

From Genetic Algorithms to LLM-Guided Evolutionary Discovery of Reinforcement Learning Algorithms: A Comprehensive Review

J.C. Vaught

CSCE 775: Deep Reinforcement Learning and Search

April 2026

Abstract

This review traces the intellectual trajectory from classical evolutionary computation to the emerging paradigm of using large language models as evolutionary operators for the automated discovery of reinforcement learning algorithms. We begin with the foundational principles of genetic algorithms, genetic programming, and evolution strategies, then examine their intersection with neural networks through neuroevolution. The exposition proceeds through the deep reinforcement learning revolution, covering value-based and policy gradient methods, trust-region optimization, and the role of RL in large language model alignment via RLHF and GRPO. We survey meta-learning and automated RL as precursors to algorithm discovery, before turning to LLM-based program synthesis. The core of the review examines LLM-guided evolutionary search frameworks, including FunSearch and AlphaEvolve, which leverage LLMs to generate and refine code within evolutionary loops. We culminate with a detailed analysis of recent work on evolving RL loss functions, particularly the REvolve framework and the Sygkounas et al. GECCO 2026 study, which demonstrates that LLM-guided evolution can produce novel, competitive RL algorithms. We conclude with a critical assessment of open problems and future directions.

Table of Contents

1. Evolutionary Computation: Foundations	4
1.1. Origins: Evolution Strategies in Germany	4
1.2. Genetic Algorithms and the Schema Theorem	4
1.3. Evolutionary Programming	5
1.4. Genetic Programming	5
1.5. Unifying the Field	5
1.6. Core Mechanisms	6
2. Neuroevolution and Evolution Strategies for Reinforcement Learning	6
2.1. NEAT and the Complexification Principle	7
2.2. Indirect Encodings and HyperNEAT	7
2.3. CoDeepNEAT and Evolving Deep Architectures	7
2.4. Evolution Strategies at Scale	8
2.5. Population-Based Training	8
2.6. AutoML-Zero and Evolving Entire Algorithms	8
2.7. Setting the Stage for LLM-Guided Evolution	9
3. Deep Reinforcement Learning: The Revolution	9
3.1. Classical Foundations	9
3.2. The DQN Breakthrough	10
3.3. Asynchronous and Actor-Critic Methods	10
3.4. Addressing Overestimation: TD3	10
3.5. Maximum Entropy RL: Soft Actor-Critic	11
3.6. Benchmarks and the Standardization of Evaluation	11
4. Policy Optimization: TRPO, PPO, and Beyond	12
4.1. Trust Region Policy Optimization	12
4.2. Generalized Advantage Estimation	12
4.3. Proximal Policy Optimization	12
4.4. Implementation Matters	13
4.5. Known Weaknesses of PPO	13
4.6. Beyond PPO: GRPO, DAPO, and CAPO	13
4.7. Synthesis	14
5. Reinforcement Learning for Large Language Model Alignment	15
5.1. Origins of RLHF	15
5.2. InstructGPT and the Three-Stage Pipeline	15
5.3. Direct Preference Optimization and Its Variants	15
5.4. Constitutional AI and AI Feedback	16
5.5. GRPO and the Elimination of the Critic	16
5.6. DeepSeek-R1 and Emergent Reasoning from Pure RL	17
5.7. RLVR and Reward Model Overoptimization	17
6. Meta-Learning and Automated Reinforcement Learning	18
6.1. Foundations of Meta-Reinforcement Learning	18
6.2. Automating the Components of RL Algorithms	18
6.3. Discovering Complete RL Algorithms	19
6.4. The AutoRL Landscape and Its Limitations	20
7. Large Language Models as Program Synthesizers	21
7.1. Foundations of LLM-Based Code Generation	21
7.2. From Code Generation to Reward Synthesis	22

7.3. The Key Insight and Its Limitations	22
8. LLM-Guided Evolutionary Search: FunSearch and AlphaEvolve	23
8.1. Foundations: Evolution Through Large Models	23
8.2. FunSearch: Mathematical Discovery via Program Search	23
8.3. AlphaEvolve: From Functions to Full Programs	24
8.4. Reflective and Adaptive Extensions	25
8.5. Evolving Game-Theoretic Algorithms	25
8.6. Synthesis	26
9. Evolving RL Algorithms: The GECCO 2026 Paper	27
9.1. Problem Statement and Motivation	27
9.2. The Evolutionary Loop	28
9.3. LLMs as Variation Operators and the Performance Gap	28
9.4. Post-Evolution Hyperparameter Optimization	29
9.5. The Discovered Algorithms	29
9.6. Empirical Results	29
9.7. Training Stability and Ablation Studies	30
9.8. Computational Cost and Broader Implications	31
10. Critical Analysis and Open Problems	31
10.1. The Novelty Question	31
10.2. The Compute Problem	32
10.3. The LLM Dependence Problem	32
10.4. The Generalization Problem	33
10.5. The Theory Vacuum	33
10.6. The Reproducibility Crisis	33
10.7. The Evaluation Bottleneck	34
10.8. Future Directions	34
References	36

1. Evolutionary Computation: Foundations

The field of evolutionary computation draws its core principles from biological evolution, encoding candidate solutions as individuals within a population and iteratively refining them through selection, recombination, and mutation. What began as independent lines of research in the United States and Germany during the 1960s and 1970s has since matured into a unified discipline with dedicated conferences, journals, and a rich theoretical literature. This section traces the intellectual origins of evolutionary algorithms, surveys the principal paradigms, and examines the theoretical tools that have shaped the field.

1.1. Origins: Evolution Strategies in Germany

The earliest experiments with evolution-inspired optimization took place at the Technical University of Berlin in the mid-1960s. Ingo Rechenberg, working alongside Hans-Paul Schwefel and Peter Bienert, sought optimal shapes for bodies in a wind tunnel and arrived at the idea of applying random perturbations to design parameters in analogy with biological mutation [1]. Rechenberg formalized this approach in his 1973 monograph *Evolutionssstrategie* (published by Frommann-Holzboog), which introduced the $(1 + 1)$ -ES, a minimal scheme in which a single parent produces a single offspring through Gaussian mutation, and the offspring replaces the parent only if it achieves superior fitness. The crucial insight was the *one-fifth success rule*, an adaptive heuristic for adjusting mutation step sizes based on the observed fraction of successful mutations.

Schwefel extended the framework substantially, introducing population-based variants with recombination operators and, most importantly, the concept of *self-adaptation*, in which strategy parameters such as mutation step sizes are themselves subject to evolution [2]. His 1981 English-language Wiley text *Numerical Optimization of Computer Models* brought evolution strategies to an international audience and established the (μ, λ) and $(\mu + \lambda)$ selection notations that remain standard today. A later survey by Beyer and Schwefel, published in the March 2002 issue of *Natural Computing*, provided a comprehensive theoretical treatment of the convergence properties and design principles underlying these methods [3].

1.2. Genetic Algorithms and the Schema Theorem

Independently, John Holland at the University of Michigan developed a different evolutionary framework motivated not primarily by engineering optimization but by the study of adaptation itself. His April 1975 book *Adaptation in Natural and Artificial Systems*, published by the University of Michigan Press, introduced the *genetic algorithm* (GA), in which candidate solutions are encoded as fixed-length binary strings and transformed through selection, crossover, and mutation operators modeled on the mechanisms of population genetics [4]. Holland's formulation emphasized the role of *schemata*, templates defined over the binary alphabet that represent subsets of the search space sharing common features at certain positions. The *Schema Theorem* provided an inequality bounding the expected number of instances of a schema in the next generation, showing that short, low-order schemata with above-average fitness tend to receive exponentially increasing representation over successive generations.

The Schema Theorem was popularized and extended by David Goldberg, Holland's student, whose January 1989 Addison-Wesley textbook *Genetic Algorithms in Search, Optimization and Machine Learning* became the field's most widely read introduction [5]. Goldberg articulated the *Building Block Hypothesis*, which posited that genetic algorithms achieve their power by assembling short, high-fitness schemata into progressively better solutions through crossover. Kenneth De Jong's August 1975 University of Michigan dissertation further cemented the role

of genetic algorithms in function optimization by establishing a standard suite of test functions and systematically studying the effects of population size, crossover rate, and mutation probability on algorithm performance [6].

However, the Schema Theorem and the Building Block Hypothesis have attracted significant criticism over the decades. The theorem is an inequality rather than an equality, and it holds exactly only in the limit of infinite populations, making it a weak predictor of finite-population behavior. Critics have observed that the theorem cannot distinguish between problem instances where genetic algorithms perform well and those where they fail, limiting its explanatory power [7]. The Building Block Hypothesis, while intuitive, has been challenged by empirical studies demonstrating that schema fitness is highly context-dependent and that the static assembly of building blocks does not reliably account for GA dynamics on deceptive or multimodal landscapes.

1.3. Evolutionary Programming

A third independent lineage originated with Lawrence Fogel in San Diego in the early 1960s. Fogel's evolutionary programming (EP) approach evolved finite-state automata to perform prediction tasks, treating the behavioral repertoire of a program rather than its syntactic structure as the object of selection, an approach laid out in the 1966 Wiley volume *Artificial Intelligence through Simulated Evolution* [8]. Unlike genetic algorithms, evolutionary programming did not employ crossover, relying instead on mutation as the sole variation operator and tournament selection as the primary survival mechanism. David Fogel later extended this line of work and placed it within the broader context of evolutionary computation in his May 1995 IEEE Press monograph *Evolutionary Computation: Toward a New Philosophy of Machine Intelligence* [7].

1.4. Genetic Programming

John Koza's December 1992 MIT Press book *Genetic Programming: On the Programming of Computers by Means of Natural Selection* introduced a paradigm in which the individuals subject to evolution are themselves computer programs, typically represented as parse trees [9]. Koza demonstrated that genetic programming (GP) could automatically discover programs solving a diverse range of problems, from symbolic regression and classification to control system design, without requiring the user to specify the form of the solution in advance. The key operators in GP are subtree crossover, which exchanges randomly selected subtrees between two parent programs, and subtree mutation, which replaces a randomly selected subtree with a newly generated random subtree. Koza's work represented the first large-scale, non-trivial demonstration of automatic programming since the concept was discussed in the 1940s, and it opened a productive research program that continues to generate novel applications in program synthesis and symbolic discovery.

1.5. Unifying the Field

By the mid-1990s, the parallel development of evolution strategies, genetic algorithms, evolutionary programming, and genetic programming had created a fragmented landscape with overlapping terminology and partially redundant theory. Thomas Back's January 1996 Oxford University Press monograph *Evolutionary Algorithms in Theory and Practice* was among the first to present all three major paradigms within a unified formal framework, comparing their selection mechanisms, representation schemes, and variation operators and identifying the common algorithmic skeleton that underlies them all [10]. Eiben and Smith's November 2003 Springer textbook *Introduction to Evolutionary Computing* further consolidated this unified

perspective for a pedagogical audience, organizing the material around the shared concepts of representation, fitness evaluation, parent selection, recombination, mutation, and survivor selection [11].

Institutional developments mirrored this intellectual convergence. The *IEEE Transactions on Evolutionary Computation*, whose inaugural issue appeared in April 1997 with David B. Fogel as editor-in-chief, provided the field’s first dedicated archival journal. Two years later, the *Genetic and Evolutionary Computation Conference* (GECCO) was established at its inaugural meeting held July 13-17, 1999 in Orlando, through the merger of the International Conference on Genetic Algorithms (ICGA, first held in July 1985 at Carnegie Mellon and continuing biennially thereafter) and the Annual Genetic Programming Conference (GP, first held in July 1996 at Stanford). GECCO was subsequently adopted by the ACM Special Interest Group on Genetic and Evolutionary Computation (SIGEVO), chartered in July 2005, completing the organizational unification of a discipline that had spent three decades developing along separate paths.

1.6. Core Mechanisms

Across all paradigms, evolutionary algorithms share a common procedural structure. A *population* of candidate solutions is initialized, typically at random, and then iteratively transformed through three operations. *Selection* determines which individuals contribute genetic material to the next generation, with mechanisms ranging from fitness-proportionate (roulette wheel) selection in classical GAs to deterministic truncation selection in evolution strategies and tournament selection in evolutionary programming. *Recombination* (or crossover) combines information from two or more parents to produce offspring, exploiting the hypothesis that fit parents share complementary partial solutions. *Mutation* introduces stochastic perturbations, maintaining diversity in the population and enabling the exploration of regions not reachable through recombination alone. The balance between *exploitation* of currently known good solutions and *exploration* of unvisited regions of the search space is the central tension governing the design of any evolutionary algorithm, and decades of theoretical and empirical work have been devoted to understanding how operator rates, population sizes, and selection pressures mediate this tradeoff.

The foundations surveyed here provide the algorithmic vocabulary and theoretical grounding upon which modern work in evolutionary discovery of reinforcement learning algorithms builds. The capacity of evolutionary methods to search large, structured, combinatorial spaces without gradient information makes them natural candidates for the meta-optimization of algorithmic components, a theme that subsequent sections of this review will develop in detail.

2. Neuroevolution and Evolution Strategies for Reinforcement Learning

The idea of evolving neural networks rather than training them with gradient descent has a long history in artificial intelligence, one that laid essential groundwork for the modern convergence of large language models and evolutionary search. This section traces the arc from early topology-evolving algorithms through large-scale evolution strategies and, ultimately, to the evolution of entire learning algorithms, highlighting both the strengths and the persistent limitations that motivated subsequent hybrid and LLM-guided approaches.

2.1. NEAT and the Complexification Principle

Stanley and Miikkulainen’s Summer 2002 Evolutionary Computation paper introduced NeuroEvolution of Augmenting Topologies (NEAT), an algorithm that evolves both the weights and the structure of neural networks simultaneously [12]. NEAT addressed three problems that had plagued earlier neuroevolution methods. First, it employed *historical markings*, unique innovation numbers assigned to each new gene as it arose through structural mutation, so that meaningful crossover between networks of differing topologies became possible without expensive topological analysis. Second, NEAT used *speciation* to protect structural innovations, grouping individuals by topological similarity and allowing novel structures time to optimize their weights before being forced to compete with the rest of the population. Third, NEAT started evolution from minimal networks and incrementally added complexity, a principle termed *complexification*, which biased search toward simple solutions and avoided the bloat common in methods that begin with large random topologies. On the challenging double pole-balancing benchmark without velocity information, NEAT achieved proficiency significantly faster than competing evolutionary algorithms, and the paper received the Outstanding Paper of the Decade award from the International Society for Artificial Life.

2.2. Indirect Encodings and HyperNEAT

A fundamental limitation of NEAT is that every connection in the network requires a separate gene, making the representation unwieldy for large networks. Stanley, D’Ambrosio, and Gauci addressed this in their April 2009 Artificial Life paper introducing HyperNEAT, which presented an *indirect encoding* based on Compositional Pattern-Producing Networks (CPPNs) [13]. Rather than specifying each weight individually, a CPPN takes as input the geometric coordinates of a source and target neuron and outputs the weight of the connection between them. Because CPPNs can express spatial regularities such as symmetry, repetition, and gradient patterns, HyperNEAT can generate connectivity for networks with millions of connections from a compact genotype. The approach was demonstrated on visual discrimination tasks where networks containing over eight million connections were evolved successfully. HyperNEAT also exhibited a form of resolution independence: the same CPPN could produce connectivity patterns at different substrate resolutions without requiring further evolution, a property with no analogue in direct-encoding methods.

2.3. CoDeepNEAT and Evolving Deep Architectures

As deep learning rose to prominence, researchers adapted neuroevolution to operate at the level of architectural components rather than individual neurons. Miikkulainen et al., in a March 2017 arXiv preprint later published as a 2019 book chapter, proposed CoDeepNEAT, a coevolutionary extension that maintained two separate populations: one of *modules*, each encoding a small subnetwork (such as a convolutional block with specific filter sizes, activation functions, and hyperparameters), and one of *blueprints*, which were graph-structured chromosomes specifying how modules should be assembled into a complete deep network [14]. By evolving modules and blueprints in tandem, CoDeepNEAT could discover architectures competitive with hand-designed networks for image classification and language modeling, and it was applied to real-world tasks such as automated image captioning. This line of work demonstrated that evolutionary methods could scale to the design of modern deep networks, though the computational cost of evaluating each candidate architecture remained substantial.

2.4. Evolution Strategies at Scale

A pivotal moment came in March 2017 when Salimans, Ho, Chen, Sidor, and Sutskever at OpenAI showed that a simple variant of evolution strategies (ES) could achieve competitive performance on standard reinforcement learning benchmarks [15]. Their approach treated the return of a policy as a black-box objective and estimated its gradient using finite differences over a population of parameter perturbations. The critical insight was a communication trick: by sharing random seeds rather than full parameter vectors, workers needed to exchange only scalar fitness values, making the method embarrassingly parallel. With 1,440 CPU cores, the system solved the MuJoCo 3D humanoid locomotion task in approximately 10 minutes, a problem that required roughly a full day for gradient-based RL methods running on comparable hardware. On Atari games, ES achieved competitive scores after about one hour of distributed training.

The excitement around ES stemmed from its simplicity and scalability. It required no back-propagation, no value function estimation, and no replay buffer. It was naturally robust to long time horizons and sparse rewards, settings where temporal-difference methods struggle. However, the approach did not ultimately replace gradient-based RL for a fundamental reason: *sample efficiency*. ES methods require far more environment interactions than policy gradient or actor-critic algorithms to reach comparable performance, because they discard the internal structure of the trajectory and treat the entire episode return as a single scalar signal [15]. In domains where environment interaction is expensive, such as robotics or complex simulation, this sample inefficiency proved prohibitive.

2.5. Population-Based Training

Rather than pitting evolution against gradient descent, Jaderberg et al. at DeepMind, in a November 2017 arXiv preprint, proposed Population-Based Training (PBT), a hybrid that used evolutionary mechanisms to optimize what gradient methods cannot: hyperparameters and training schedules [16]. PBT maintains a population of agents, each training independently via standard gradient-based methods but with different hyperparameter configurations. Periodically, underperforming agents copy the weights of better-performing ones (exploitation) and then perturb their hyperparameters (exploration). This procedure discovers *schedules* of hyperparameter settings over the course of training rather than a single fixed configuration, capturing the intuition that the optimal learning rate or entropy coefficient may change as training progresses. PBT demonstrated improvements in deep reinforcement learning, machine translation, and generative adversarial network training, establishing that the most effective use of evolutionary ideas in deep learning might be at the meta-level rather than at the level of raw parameter optimization.

2.6. AutoML-Zero and Evolving Entire Algorithms

The most ambitious application of neuroevolution to date is AutoML-Zero, introduced by Real, Liang, So, and Le at Google Brain in a March 2020 arXiv preprint later presented at ICML in July 2020 [17]. Rather than evolving network architectures or hyperparameters, AutoML-Zero evolved complete machine learning algorithms from scratch, starting from populations of empty programs and using only basic mathematical operations (addition, multiplication, matrix operations) as primitives. Through regularized tournament selection and mutation, the system independently rediscovered two-layer neural networks trained with backpropagation, as well as linear regression, demonstrating that evolution can navigate a search space estimated at 10^{12} possible programs to find needle-in-a-haystack solutions. The significance of this result

was conceptual as much as practical: it established that the space of learning algorithms is itself amenable to automated search, provided the search process is sufficiently powerful.

2.7. Setting the Stage for LLM-Guided Evolution

Collectively, these works established several principles that inform the current generation of LLM-guided evolutionary methods. NEAT and HyperNEAT demonstrated that evolving structure alongside parameters produces more capable solutions than optimizing either alone. CoDeepNEAT showed that coevolution of modular components enables scaling to modern architectures. ES at scale proved that evolutionary search can be massively parallelized but also exposed the fundamental sample-efficiency gap relative to gradient methods. PBT demonstrated the power of evolving at the meta-level, optimizing the training process rather than the parameters directly. AutoML-Zero showed that evolution can discover algorithms, not merely optimize them. The natural next question, which subsequent sections of this survey address, is whether large language models, with their capacity for code generation, semantic understanding, and in-context reasoning, can serve as more intelligent mutation and crossover operators than the random perturbations used throughout the neuroevolution literature.

3. Deep Reinforcement Learning: The Revolution

The synthesis of deep neural networks with reinforcement learning transformed the field from a collection of elegant but limited tabular algorithms into a general-purpose framework capable of mastering complex, high-dimensional tasks. This section traces that transformation, beginning with the classical foundations of temporal-difference learning and Q-learning, proceeding through the deep Q-network breakthrough, and arriving at the modern actor-critic algorithms that now serve as standard baselines for continuous control.

3.1. Classical Foundations

The intellectual roots of deep reinforcement learning lie in the temporal-difference (TD) methods formalized by Sutton [18]. TD learning introduced the idea of updating value estimates based on the difference between temporally successive predictions rather than waiting for a final outcome, enabling incremental, online learning in environments where outcomes are delayed. This principle proved foundational to nearly every algorithm that followed. Building on TD methods, Watkins [19] proposed Q-learning, an off-policy algorithm that learns the optimal action-value function $Q^*(s, a)$ directly, without requiring a model of the environment’s dynamics. Watkins and Dayan [20] subsequently proved that Q-learning converges to Q^* with probability one, provided all state-action pairs are visited infinitely often and the learning rates satisfy the Robbins-Monro conditions. These convergence guarantees, however, applied only to the tabular setting. The comprehensive treatment by Sutton and Barto, first published in March 1998 and substantially revised in November 2018 [21], remains the standard reference for these foundations, presenting TD learning, Monte Carlo methods, and dynamic programming as a unified framework for sequential decision-making under uncertainty.

The policy gradient theorem, established by Sutton, McAllester, Singh, and Mansour [22], opened a parallel line of research by demonstrating that the gradient of expected cumulative reward with respect to policy parameters can be expressed in a form amenable to estimation from sampled trajectories. Specifically, the theorem shows that $\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)]$, where π_{θ} is a parameterized stochastic policy and $Q^{\pi_{\theta}}$ is the corresponding action-value function. This result provided the theoretical foundation for REIN-

FORCE, actor-critic methods, and the entire family of policy optimization algorithms that would later prove essential for continuous action spaces.

3.2. The DQN Breakthrough

The deep Q-network (DQN), introduced by Mnih et al. in a December 2013 arXiv preprint [23] and refined in their landmark Nature paper in February 2015 [24], demonstrated for the first time that a single reinforcement learning agent could learn to play a diverse collection of Atari 2600 games directly from raw pixel inputs. The key architectural insight was to represent the action-value function $Q(s, a; \theta)$ with a convolutional neural network, taking stacked frames as input and outputting Q-values for each discrete action. Two algorithmic innovations proved essential for stable training. First, experience replay, originally proposed by Lin [25], stores transitions (s_t, a_t, r_t, s_{t+1}) in a fixed-size buffer and samples mini-batches uniformly at random, breaking the temporal correlations that destabilize stochastic gradient descent on sequentially collected data. Second, a separate target network with parameters θ^- , updated periodically from the online network, provides stable regression targets $r + \gamma \max_{a'} Q(s', a'; \theta^-)$ and prevents the feedback loop in which the network chases its own shifting predictions. Evaluated on the Arcade Learning Environment [26], DQN achieved human-level performance on the majority of 49 tested games using identical hyperparameters and architecture across all domains, establishing the first convincing demonstration that deep reinforcement learning could scale to high-dimensional sensory inputs.

3.3. Asynchronous and Actor-Critic Methods

While DQN demonstrated the viability of deep RL in discrete action spaces, scaling to continuous control and improving training efficiency required new paradigms. Mnih et al. [27], in work released on arXiv in February 2016 and presented at ICML in June 2016, proposed the Asynchronous Advantage Actor-Critic (A3C), which replaced the replay buffer with multiple parallel actor-learner threads, each interacting with its own copy of the environment. By aggregating gradients asynchronously, A3C achieved a decorrelation effect analogous to experience replay while enabling on-policy learning. The advantage function $A(s, a) = Q(s, a) - V(s)$ reduced variance in the policy gradient estimates. A3C surpassed DQN’s Atari performance while training in half the time on a single multi-core CPU, demonstrating that parallelism could substitute for the memory overhead of replay buffers.

For continuous action spaces, Lillicrap et al. [28], in work released on arXiv in September 2015 and presented at ICLR in May 2016, introduced the Deep Deterministic Policy Gradient (DDPG), which combined the deterministic policy gradient with DQN-style innovations. DDPG maintains an actor network $\mu(s, \theta^\mu)$ that outputs a continuous action and a critic network $Q(s, a, \theta^Q)$ that evaluates it, using experience replay and target networks for stability. The algorithm solved over 20 simulated physics tasks from simple balance problems to complex locomotion, establishing the actor-critic template that subsequent continuous-control algorithms would refine. However, DDPG exhibited well-documented brittleness to hyperparameter choices, often requiring careful tuning to avoid divergence caused by overestimation in the learned Q-function.

3.4. Addressing Overestimation: TD3

Fujimoto, van Hoof, and Meger [29], in work posted to arXiv in February 2018 and presented at ICML in July 2018, diagnosed overestimation bias as the primary failure mode of DDPG and proposed Twin Delayed Deep Deterministic Policy Gradient (TD3) to address it through three mechanisms. First, clipped double Q-learning maintains two independent critic

networks and uses the minimum of their predictions to form the Bellman target, $y = r + \gamma \min_{i=1,2} Q(s', \tilde{a}; \theta_i^-)$, systematically suppressing the positive bias that accumulates when a single critic’s errors are exploited by the policy. Second, delayed policy updates reduce the frequency of actor updates relative to critic updates, allowing the value estimates to stabilize before the policy adapts to them. Third, target policy smoothing adds clipped noise to the target action, regularizing the critic by preventing it from developing sharp peaks that the actor could exploit. Together, these modifications produced consistent improvements across the MuJoCo benchmark suite [30], outperforming DDPG in every environment tested.

3.5. Maximum Entropy RL: Soft Actor-Critic

Haarnoja et al. [31], in work posted to arXiv in January 2018 and presented at ICML in July 2018, reframed the reinforcement learning objective itself by augmenting the standard expected return with an entropy bonus, yielding the maximum entropy objective $J(\pi) = \sum_t \mathbb{E}_\pi[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$, where α is a temperature parameter controlling the trade-off between reward maximization and stochastic exploration. The resulting Soft Actor-Critic (SAC) algorithm inherits the off-policy efficiency of DDPG while gaining several advantages from the entropy regularization. The stochastic policy explores more broadly, the optimization landscape is smoother, and the algorithm is substantially more robust to hyperparameter choices. In a follow-up paper, Haarnoja et al. [32] introduced automatic temperature tuning by casting the temperature selection as a constrained optimization problem, eliminating the most sensitive hyperparameter and making SAC nearly turnkey for new domains. SAC rapidly became the de facto standard for continuous control benchmarks, achieving state-of-the-art performance on MuJoCo tasks [30] and the DeepMind Control Suite [33] while demonstrating successful transfer to real-world robotic manipulation and locomotion.

3.6. Benchmarks and the Standardization of Evaluation

The rapid progress described above was enabled in no small part by the development of standardized evaluation platforms. The Arcade Learning Environment [26] provided a uniform interface to over a hundred Atari 2600 games, each presenting distinct challenges in exploration, credit assignment, and partial observability. For continuous control, the MuJoCo physics engine [30] enabled the creation of a canonical suite of locomotion and manipulation tasks that became the primary testbed for algorithms like DDPG, TD3, and SAC. The DeepMind Control Suite [33] later offered a curated, standardized set of continuous-control tasks with interpretable reward functions built on MuJoCo, further facilitating reproducible comparison. These benchmarks collectively established the empirical methodology of the field, providing the common ground on which algorithmic claims could be evaluated and contested.

4. Policy Optimization: TRPO, PPO, and Beyond

The central challenge of policy gradient methods is that naive gradient ascent on the expected return can produce catastrophically large policy updates, destabilizing training and erasing previously learned behavior. The line of work from Trust Region Policy Optimization through Proximal Policy Optimization and its successors represents a sustained effort to tame this instability while keeping the algorithms practical at scale.

4.1. Trust Region Policy Optimization

The theoretical foundation for modern policy optimization was laid by Kakade and Langford at ICML in July 2002 [34], who showed that a conservative policy iteration scheme could guarantee monotonic improvement by bounding the divergence between successive policies. Schulman, Levine, Moritz, Jordan, and Abbeel [35] extended this insight into a practical deep reinforcement learning algorithm, Trust Region Policy Optimization (TRPO), first released on arXiv in February 2015 and presented at ICML in July 2015. The key idea is to maximize a surrogate objective, the expected advantage of the new policy under the state distribution of the old policy, subject to a constraint that the KL divergence between the old and new policies remains below a threshold δ . Formally, at each iteration TRPO solves $\max_{\theta} \hat{\mathbb{E}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}(s, a) \right]$ subject to $\hat{\mathbb{E}} \left[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) \right] \leq \delta$. This constrained optimization is solved approximately using the conjugate gradient method, which computes the natural gradient direction by solving a linear system involving the Fisher information matrix without ever forming or storing that matrix explicitly [35]. The resulting update direction is then rescaled via a line search to satisfy the KL constraint. TRPO provided the first algorithm for deep policy optimization with a principled monotonic improvement guarantee, and it demonstrated strong empirical performance on continuous control benchmarks. However, the conjugate gradient computation and line search introduced significant computational overhead and implementation complexity, limiting its adoption in large-scale settings.

4.2. Generalized Advantage Estimation

A critical companion to trust-region methods is the question of how to estimate the advantage function $A(s, a)$ that appears in the policy gradient. Schulman, Moritz, Levine, Jordan, and Abbeel [36] proposed Generalized Advantage Estimation (GAE) in June 2015 on arXiv, with the refined version presented at ICLR in May 2016, an exponentially weighted estimator that interpolates between high-bias, low-variance estimates (relying heavily on a learned value function) and low-bias, high-variance estimates (relying on full Monte Carlo returns). GAE introduces a parameter $\lambda \in [0, 1]$ that controls this tradeoff. When $\lambda = 0$, the estimator reduces to the one-step temporal difference residual, which has low variance but can be biased if the value function is inaccurate. When $\lambda = 1$, it recovers the full Monte Carlo advantage, which is unbiased but high variance. In practice, values of λ between 0.9 and 0.99 are typical. The discount factor γ introduces a separate, complementary bias-variance tradeoff. Taking $\gamma < 1$ introduces bias regardless of the value function’s accuracy, while $\lambda < 1$ introduces bias only when the value function is imperfect [36]. GAE became a standard component in nearly all subsequent policy optimization algorithms, including PPO and its descendants.

4.3. Proximal Policy Optimization

Schulman, Wolski, Dhariwal, Radford, and Klimov [37] introduced Proximal Policy Optimization (PPO) on July 20, 2017, alongside an OpenAI blog post the same day, as a dramatically simpler alternative to TRPO. Rather than solving a constrained optimization problem with conjugate gradients, PPO enforces the trust region implicitly through a clipped surrogate

objective. The probability ratio $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is clipped to the interval $[1 - \epsilon, 1 + \epsilon]$, and the objective takes the pessimistic (lower) bound of the clipped and unclipped terms. The resulting objective is $L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$. This formulation removes the need for conjugate gradient solvers, Fisher information matrices, and line searches. PPO can be implemented in a few dozen lines of code and optimized with standard stochastic gradient ascent over multiple epochs of minibatch updates on the same batch of collected trajectories. The clipping mechanism prevents the policy from changing too rapidly in any single update, achieving a similar effect to the KL constraint in TRPO but with far less computational cost.

PPO’s simplicity, stability, and scalability made it the dominant policy optimization algorithm across robotics, game-playing, and, most consequentially, the reinforcement learning from human feedback (RLHF) pipelines used to align large language models [37]. OpenAI’s adoption of PPO for training ChatGPT, released in November 2022, cemented its status as the default choice for practitioners.

4.4. Implementation Matters

Despite PPO’s apparent algorithmic elegance, a growing body of evidence suggests that its empirical dominance owes as much to engineering as to the clipped objective itself. Engstrom, Ilyas, Santurkar, Tsipras, Janoos, Rudolph, and Madry [38], in work presented at ICLR in April 2020, conducted a systematic ablation study and found that code-level optimizations, details present only in reference implementations and not described in the original paper, are responsible for most of PPO’s gain in cumulative reward over TRPO. These optimizations include global gradient clipping (constraining the L2 norm of the gradient to not exceed 0.5), value function clipping, reward normalization, observation normalization, orthogonal initialization, and learning rate annealing. When these implementation details were added to TRPO, the performance gap between the two algorithms largely vanished [38]. Huang, Dossa, Raffin, Kanervisto, and Wang [39], in an ICLR blog post from March 2022, subsequently cataloged 37 distinct implementation details across different PPO application domains, further underscoring the gap between the algorithm as published and the algorithm as practiced. These findings raise an uncomfortable question for the field. If the theoretical contribution of the clipped surrogate is secondary to engineering details that accumulated through trial and error, then PPO’s dominance may be partly an artifact of implementation maturity rather than algorithmic superiority.

4.5. Known Weaknesses of PPO

PPO’s weaknesses are well documented. As an on-policy method, it must discard collected trajectories after each policy update, leading to poor sample efficiency in environments where interaction is expensive. The clipping parameter ϵ , learning rate, entropy bonus coefficient, number of minibatch epochs, and batch size all require careful tuning, and the sensitivity of performance to these hyperparameters makes reproducibility difficult. The conservative nature of the clipped update, while stabilizing, can also slow convergence and cause the policy to become trapped in local optima, particularly in high-dimensional or sparse-reward settings. Gaussian exploration noise, the default in continuous control, can be insufficient for complex state spaces, leading to inadequate exploration [37].

4.6. Beyond PPO: GRPO, DAPO, and CAPO

The explosion of interest in training large language models for reasoning has driven a new generation of policy optimization algorithms that build on PPO’s foundations while addressing

its specific limitations in the LLM setting. Group Relative Policy Optimization (GRPO), introduced by Shao, Wang, Zhu, and Xu in the DeepSeekMath work released on arXiv in February 2024 [40], eliminates the critic network entirely. Instead of training a separate value function to estimate baselines, GRPO generates a group of candidate responses for each prompt and computes advantages relative to the group’s mean reward. This reduces memory overhead by approximately 50% compared to PPO, a critical consideration when the policy itself is a multi-billion-parameter language model. GRPO retains the clipped surrogate objective from PPO but replaces the learned baseline with a sample-based one. DeepSeek-R1, released in January 2025, demonstrated that GRPO could train competitive reasoning models from scratch using only reinforcement learning with verifiable rewards [41].

DAPO (Decoupled Clip and Dynamic Sampling Policy Optimization), developed by Yu, Zhang, Zhu, Yuan, and colleagues at ByteDance Seed and Tsinghua University and released on arXiv in March 2025 [42], addresses the entropy collapse problem that arises during long chain-of-thought reinforcement learning. DAPO decouples the lower and upper clipping bounds, increasing the upper bound ϵ_{high} to allow low-probability tokens more room to increase in likelihood, thereby preserving diversity in the policy’s output distribution. Dynamic sampling filters out prompts for which the current policy achieves either perfect or zero accuracy, ensuring that every training batch contains prompts with effective gradient signal. Token-level policy gradient loss and overlong reward shaping further stabilize training. DAPO achieved 50% accuracy on AIME 2024 with Qwen2.5-32B, surpassing DeepSeek-R1-Zero-Qwen-32B while requiring 50% fewer training steps [42].

Most recently, Melo, Abate, and Gal proposed Curvature-Aware Policy Optimization (CAPO), published at ICLR in April 2026 [43]. CAPO exploits an inexpensive approximation of second-order curvature information to identify and mask out tokens whose gradient contributions would destabilize the update. The method achieves up to a 30-fold improvement in sample efficiency over standard GRPO on mathematical reasoning benchmarks, with minimal intervention, rejecting fewer than 8% of tokens and adding negligible computational overhead. CAPO also provides monotonic improvement guarantees under practical assumptions, reconnecting modern LLM-focused policy optimization with the theoretical tradition of TRPO. Extensions to other base algorithms, including Dr.GRPO and REINFORCE, yield Dr.CAPO and ReinCAPO, demonstrating the generality of curvature-aware masking [43].

4.7. Synthesis

The trajectory from TRPO through PPO and into GRPO, DAPO, and CAPO reveals a field in tension between theoretical rigor and engineering pragmatism. TRPO offered monotonic improvement guarantees but was too complex for widespread adoption. PPO traded theoretical elegance for simplicity and became dominant, yet its success depended heavily on implementation details that were never part of the published algorithm [38]. The LLM era has forced a rethinking of these tradeoffs. Critic-free methods like GRPO reduce memory costs, entropy-aware methods like DAPO prevent mode collapse during long-horizon reasoning, and curvature-aware methods like CAPO recover sample efficiency and stability guarantees without the computational burden of full second-order optimization. Whether the next generation of policy optimization algorithms will achieve a cleaner separation between algorithmic contribution and engineering artifact remains an open question.

5. Reinforcement Learning for Large Language Model Alignment

The application of reinforcement learning to the alignment of large language models represents one of the most consequential transfers of RL methodology from its traditional game-playing and robotics domains into natural language processing. The central problem is straightforward in motivation but formidable in execution. A pretrained language model possesses broad knowledge and fluent generation capabilities, yet it has no intrinsic mechanism for distinguishing helpful, truthful, and safe outputs from harmful or misleading ones. Reinforcement learning provides the optimization machinery to steer model behavior toward human-preferred outputs, and the story of how this machinery has evolved over the past decade is one of progressive simplification, from heavyweight actor-critic pipelines to elegant closed-form solutions.

5.1. Origins of RLHF

The intellectual foundation for reinforcement learning from human feedback was laid by Christiano et al. [44], posted to arXiv in June 2017 and presented at NeurIPS in December 2017, who demonstrated that complex RL tasks in Atari and simulated robotics could be solved using human preference comparisons rather than hand-specified reward functions. Earlier precursors such as the TAMER framework of Knox and Stone in 2009 had already explored interactive shaping from human reinforcement signals, but the 2017 work brought the paradigm into the deep RL era. Christiano’s method maintained two learned components, a policy trained via standard RL and a reward model trained on human comparisons between trajectory segment pairs. Critically, the approach required feedback on less than one percent of the agent’s environment interactions, establishing the practical feasibility of human-in-the-loop reward learning. Stiennon et al. [45], released on arXiv in September 2020 and appearing at NeurIPS in December 2020, extended this paradigm to natural language, showing that a reward model trained on human comparisons between summaries could guide policy optimization to produce outputs that human evaluators preferred over both supervised baselines and human-written reference summaries.

5.2. InstructGPT and the Three-Stage Pipeline

The landmark contribution of Ouyang et al. [46], posted to arXiv in March 2022, was to scale RLHF to large language models, producing InstructGPT. This work built directly on the GPT-3 base model released by OpenAI in May 2020, and its techniques later underpinned the ChatGPT release on November 30, 2022 and the GPT-4 release on March 14, 2023. Their three-stage pipeline became the template for an entire generation of aligned models. In the first stage, a pretrained GPT-3 model is fine-tuned via supervised learning on high-quality human demonstrations of instruction-following, yielding a supervised fine-tuned (SFT) model. In the second stage, human labelers rank multiple model outputs for a given prompt, and these ranked comparisons train a reward model that learns to predict human preferences. In the third stage, Proximal Policy Optimization (PPO) optimizes the SFT model against the learned reward signal, with a KL-divergence penalty to prevent the policy from deviating too far from the SFT baseline. The results were striking. A 1.3 billion parameter InstructGPT model was preferred by human evaluators over the 175 billion parameter GPT-3, demonstrating that alignment training could be far more efficient than simply scaling model size.

5.3. Direct Preference Optimization and Its Variants

Despite the success of the PPO-based pipeline, its complexity motivated the search for simpler alternatives. Rafailov et al. [47], in work posted to arXiv in May 2023 and presented at NeurIPS

in December 2023, introduced Direct Preference Optimization (DPO), which eliminated the reward model entirely through an elegant reparameterization. The key insight was that the optimal policy under the RLHF objective can be expressed in closed form as a function of the reward, and conversely, the reward can be expressed in terms of the optimal policy and a reference policy. By substituting this reparameterized reward into the Bradley-Terry preference model, the intractable partition function cancels, yielding a simple binary cross-entropy loss defined directly over preference pairs. DPO thus collapses the three-stage RLHF pipeline into a single-stage classification problem, matching or exceeding PPO-based RLHF on tasks ranging from sentiment control to summarization and dialogue.

The simplicity of the DPO framework catalyzed a rapid proliferation of variants. Azar et al. [48] proposed Identity Preference Optimization (IPO), which replaces the logistic loss with a squared loss and incorporates a margin term, addressing the observation that DPO’s implicit reward gap between chosen and rejected responses can grow unboundedly during training, driving the policy toward extreme probability assignments. Ethayarajh et al. [49] introduced Kahneman-Tversky Optimization (KTO), inspired by prospect theory, which maximizes the utility of generations directly and requires only binary accept/reject labels rather than paired comparisons, dramatically simplifying data collection. Meng et al. [50] proposed SimPO, a reference-model-free approach that adopts length-normalized likelihoods and a constant margin to separate preferred from dispreferred responses, further reducing computational overhead. Each variant addresses a specific limitation of the original DPO formulation, whether in training stability, data requirements, or computational cost, while preserving the core advantage of avoiding explicit reward model training.

5.4. Constitutional AI and AI Feedback

An alternative approach to reducing reliance on human labelers was proposed by Bai et al. [51] through Constitutional AI, posted to arXiv in December 2022 by Anthropic and subsequently used to train the Claude family of models beginning with Claude 1 in March 2023, Claude 2 in July 2023, and Claude 3 in March 2024. Rather than collecting human preference labels for harmlessness training, the method uses a set of written principles, a “constitution,” to guide an AI system in critiquing and revising its own outputs. In the supervised phase, the model generates responses, self-critiques them against constitutional principles, and produces revised outputs on which the model is then fine-tuned. In the RL phase, the model evaluates pairs of its own outputs to generate preference labels, and a reward model is trained on these AI-generated comparisons. This approach, termed Reinforcement Learning from AI Feedback (RLAIF) and further studied by Lee et al. at Google in work posted to arXiv in September 2023, demonstrated that harmless behavior could be induced with minimal human oversight while avoiding the evasiveness that often accompanies safety-focused training.

5.5. GRPO and the Elimination of the Critic

Group Relative Policy Optimization (GRPO), introduced by Shao et al. [40] in the DeepSeek-Math work posted to arXiv on February 5, 2024, represents a different axis of simplification targeting the RL training itself rather than the preference modeling. Standard PPO requires a critic (value) network to estimate the advantage function, which roughly doubles the memory and compute requirements of training. GRPO eliminates the critic entirely by sampling a group of multiple outputs for each prompt and using the group-level statistics, specifically the mean and standard deviation of reward scores within each group, to compute relative advantages. This group-relative normalization provides a stable baseline without learning a separate value

function, achieving substantial improvements on mathematical reasoning benchmarks while dramatically reducing resource requirements.

5.6. DeepSeek-R1 and Emergent Reasoning from Pure RL

The most dramatic demonstration of RL-driven alignment came with DeepSeek-R1 [41], released in January 2025. The DeepSeek-R1-Zero experiment applied GRPO directly to a base language model without any supervised fine-tuning, using only rule-based rewards for answer correctness and output formatting. The results were remarkable. On the AIME 2024 mathematics benchmark, pass@1 accuracy rose from 15.6% to 71.0% over the course of training, with majority voting pushing performance to 86.7%. More striking than the quantitative gains were the emergent behaviors: the model spontaneously developed self-verification, reflection, and extended chains of thought. The researchers documented what they termed an “aha moment,” in which the model learned to pause, reconsider its reasoning, and correct errors using an almost anthropomorphic tone of self-discovery. While the pure RL approach suffered from readability issues and language mixing, these findings demonstrated that sophisticated reasoning strategies could emerge from RL optimization alone, without being explicitly taught through supervised demonstrations. The full DeepSeek-R1 pipeline incorporated cold-start SFT data to mitigate these issues while preserving the emergent capabilities.

5.7. RLVR and Reward Model Overoptimization

The success of DeepSeek-R1 has intensified interest in Reinforcement Learning with Verifiable Rewards (RLVR), a term coined by Lambert et al. [41] in the Tulu 3 work released on arXiv in November 2024. RLVR sidesteps the learned reward model entirely for domains where correctness can be checked programmatically, such as mathematics and code generation, using deterministic verifiers as the reward signal. The Tulu 3 pipeline combined SFT, DPO, and RLVR to produce models that surpassed the instruction-tuned versions of Llama 3.1, released by Meta in July 2024 as successor to the Llama 2 release of July 2023, Qwen 2.5, and even closed models such as GPT-4o-mini on certain benchmarks.

The motivation for verifiable rewards is sharpened by the problem of reward model overoptimization, characterized formally by Gao et al. [52] in work posted to arXiv in October 2022 and presented at ICML in July 2023. Their work established scaling laws for the phenomenon in which optimizing too aggressively against a learned proxy reward model leads to degraded performance on the true objective, a manifestation of Goodhart’s law. They showed that the relationship between proxy reward optimization and ground truth performance follows predictable functional forms whose coefficients scale smoothly with reward model size, providing practitioners with quantitative guidance on when to stop optimization. This fundamental tension between reward model fidelity and optimization pressure remains one of the central challenges in RLHF-based alignment, and the shift toward verifiable rewards for suitable domains represents a partial but principled resolution.

6. Meta-Learning and Automated Reinforcement Learning

The design of reinforcement learning algorithms has historically been a manual, iterative process in which researchers propose update rules, loss functions, and hyperparameter schedules on the basis of theoretical intuition and extensive empirical tuning. Meta-learning, often described as “learning to learn,” offers an alternative paradigm in which an outer optimization loop discovers the learning algorithm itself, while an inner loop applies the discovered algorithm to solve individual tasks. This section traces the evolution of meta-reinforcement learning from its earliest formulations through to modern systems capable of discovering update rules that rival hand-designed methods across large-scale benchmarks. We also situate these advances within the broader landscape of Automated Reinforcement Learning (AutoRL) and discuss the limitations that ultimately motivated the turn toward large language model-based algorithm search.

6.1. Foundations of Meta-Reinforcement Learning

Two concurrent works in November 2016 established the meta-RL paradigm. Duan et al. [53] proposed `RLsuper`[2] in November 2016, in which a recurrent neural network (RNN) receives observations, actions, rewards, and termination signals as input, retaining its hidden state across episodes within a given Markov decision process. The weights of the RNN are trained slowly via a general-purpose RL algorithm across a distribution of tasks, while the activations of the network encode a fast adaptation procedure for each new task encountered at test time. Evaluated on multi-armed bandits and tabular MDPs, `RLsuper`[2] approached the performance of Bayes-optimal strategies without any explicit Bayesian computation. Independently, Wang et al. [54] at DeepMind introduced a closely related framework in “Learning to Reinforcement Learn,” also posted in November 2016, demonstrating through seven proof-of-concept experiments that a recurrent meta-learner trained with one RL algorithm can implement an entirely separate learned RL procedure in its recurrent dynamics, one configured to exploit structure in the training domain. Together, these two papers showed that the boundary between “algorithm” and “learned behavior” could be dissolved: the policy network does not merely solve tasks but implements the learning algorithm itself.

Model-Agnostic Meta-Learning (MAML), introduced by Finn, Abbeel, and Levine in March 2017 [55] and presented at ICML in August 2017, took a complementary approach by operating directly in parameter space rather than encoding adaptation in recurrent activations. MAML optimizes a neural network’s initial parameters such that a small number of gradient steps on a new task’s loss yields strong generalization. Because MAML makes no assumptions about the model architecture, it applies uniformly to supervised classification, regression, and reinforcement learning. Its elegance, however, comes at a computational cost: the outer loop requires differentiating through the inner optimization, producing second-order gradients that scale poorly with the number of inner steps and parameters. This tension between generality and tractability has been a recurring theme in meta-learning research.

6.2. Automating the Components of RL Algorithms

Whereas `RLsuper`[2], Wang et al., and MAML meta-learn entire adaptation procedures or initializations, subsequent work asked a more targeted question: can individual components of an RL algorithm, such as the discount factor, the bootstrapping parameter, or the loss function itself, be learned rather than hand-specified? Xu, van Hasselt, and Silver [56] introduced meta-gradient RL in May 2018, presented at NeurIPS in December 2018, in which the return function is treated as a parametric object whose meta-parameters, including the discount factor γ and

the bootstrapping parameter λ , are adjusted online via gradient descent on a meta-objective. By differentiating the agent’s performance with respect to these meta-parameters, the algorithm dynamically adapts the nature of the return over the course of training, yielding improvements on the Atari benchmark without manual tuning of these quantities.

Houthoofd et al. [57] proposed Evolved Policy Gradients (EPG) in February 2018, later presented at NeurIPS in December 2018, which meta-learns a differentiable loss function parameterized by temporal convolutions over the agent’s experience. Rather than backpropagating through the inner loop, EPG uses evolution strategies to optimize the meta-loss, sidestepping the second-order gradient difficulties of MAML-style approaches. The resulting evolved loss enabled faster learning on randomized continuous control tasks and demonstrated superior out-of-distribution generalization compared to RL^{super}[2] and MAML, though it was outperformed by RL^{super}[2] on in-distribution tasks. Kirsch, van Steenkiste, and Schmidhuber [58] extended this direction in October 2019 with MetaGenRL, later presented at ICLR in April 2020, which distills the experiences of many agents to meta-learn a compact neural objective function. By leveraging off-policy second-order gradients during meta-training, MetaGenRL achieved strong sample efficiency and, notably, generalized to environments that were entirely different from those seen during meta-training, in some cases outperforming human-engineered RL algorithms.

6.3. Discovering Complete RL Algorithms

The most ambitious line of meta-RL research aims not merely to tune components but to discover entire RL algorithms from scratch. Oh et al. [59] introduced Learned Policy Gradient (LPG) at NeurIPS in December 2020, in which a neural network parameterizes the loss function used to update the agent’s policy. The meta-learner jointly discovers what the agent should predict and how those predictions should drive policy improvement. Remarkably, LPG independently rediscovered an analogue of value functions and a bootstrapping mechanism, and when trained solely on simple toy environments, generalized to achieve non-trivial performance on complex Atari games.

Co-Reyes et al. [60] at Google Brain approached the same goal from a different angle at ICLR in May 2021 with “Evolving Reinforcement Learning Algorithms.” Rather than parameterizing loss functions with neural networks, they represented the loss as a symbolic computational graph and applied regularized evolution, a variant of genetic programming, to search over graph structures. When learning from scratch on simple gridworld and classical control tasks, the method independently rediscovered temporal-difference learning. When bootstrapped from DQN, the evolved algorithms generalized across classical control, gridworld, and Atari domains, offering the additional advantage of producing interpretable, closed-form loss expressions.

The culmination of this line of work is DiscoRL, published by Oh et al. in Nature in October 2025 [61]. Building on the LPG framework, DiscoRL meta-learns update rules from the cumulative experience of a population of agents across large-scale environment distributions. Two principal algorithms emerged from this process. Disco57, trained on the Atari-57 suite, generalized zero-shot to unseen ProcGen and DMLab-30 domains. Disco103, trained on the expanded union of Atari-57, ProcGen, and DMLab-30, was further evaluated on entirely held-out benchmarks including Crafter, NetHack, and Sokoban. Both variants surpassed all existing hand-designed algorithms on the Atari benchmark, outperforming Rainbow DQN, DQN, PPO, and A3C, methods that required decades of human ingenuity to develop. DiscoRL additionally matched MuZero’s final performance with approximately 40% less computation, establishing that meta-learned algorithms can compete with the most sophisticated planning-based approaches.

6.4. The AutoRL Landscape and Its Limitations

Parker-Holder et al. [62] provided a comprehensive survey of the Automated Reinforcement Learning (AutoRL) field in January 2022, proposing a taxonomy that spans hyperparameter optimization, neural architecture search, algorithm selection, and the meta-learning of update rules discussed above. The survey highlighted both the promise and the fragility of RL: agent performance is often acutely sensitive to design choices, and the combinatorial space of possible configurations makes manual tuning impractical at scale.

Despite the impressive results of systems like DiscoRL, the meta-learning paradigm for algorithm discovery faces persistent limitations. First, meta-learned algorithms tend to overfit to their training task distribution. Disco57 generalizes well to ProcGen and DMLab-30, but the extent to which any meta-learned rule can transfer to fundamentally novel problem structures remains an open question. Second, the computational cost is extreme: training DiscoRL requires running populations of agents across hundreds of environments for billions of frames, placing it beyond the reach of most research groups. Third, neural-network-parameterized update rules are opaque. Unlike the symbolic expressions produced by Co-Reyes et al. [60], the learned rules of LPG and DiscoRL resist human interpretation, making it difficult to extract scientific insight or to diagnose failure modes. These three challenges, distributional overfitting, prohibitive compute, and opacity of discovered algorithms, collectively motivated the search for alternative discovery mechanisms. The emergence of large language models as program synthesizers, capable of generating and refining readable code, offered a promising resolution: algorithms expressed as human-interpretable source code, searched over by LLM-guided evolutionary processes that leverage natural language reasoning rather than opaque gradient-based meta-optimization.

7. Large Language Models as Program Synthesizers

The emergence of large language models trained on massive corpora of source code has fundamentally altered the landscape of program synthesis. Rather than relying on hand-crafted grammars or domain-specific languages, modern code-generating models learn statistical regularities across billions of lines of human-written code, enabling them to synthesize functionally correct programs from natural language specifications. This capability has profound implications for reinforcement learning research, where the ability to automatically generate reward functions, loss functions, and policy components can accelerate algorithmic discovery beyond what manual engineering or random search can achieve.

7.1. Foundations of LLM-Based Code Generation

The watershed moment for neural program synthesis arrived in July 2021 with Codex, a 12-billion parameter GPT-3 model fine-tuned on publicly available code from GitHub [63]. Evaluated on HumanEval, a benchmark of 164 hand-crafted programming problems measuring functional correctness from docstrings, Codex achieved a 28.8% pass@1 rate, compared to 0% for the unmodified GPT-3. With repeated sampling of 100 candidates per problem, performance rose to 70.2%, demonstrating that even imperfect generators become powerful when paired with selection mechanisms [63]. Codex served as the backbone of GitHub Copilot, which entered technical preview in June 2021 and reached general availability in June 2022, bringing LLM-based code generation into mainstream software engineering and establishing that language models could serve as practical programming assistants.

DeepMind’s AlphaCode, released in February 2022 and subsequently published in *Science* in December 2022, extended neural code generation to the significantly harder domain of competitive programming [64]. The system adopted a generate-and-filter paradigm at unprecedented scale, producing up to millions of candidate solutions per problem using a large transformer pre-trained on GitHub code and fine-tuned on competitive programming datasets. A filtering pipeline then removed approximately 99% of generated samples based on execution against example test cases, followed by clustering of remaining solutions by program behavior to select a final set of at most 10 submissions [64]. In simulated evaluations on the Codeforces platform, AlphaCode achieved an average ranking in the top 54.3% of human competitors. The significance of this result lies not in superhuman performance but in demonstrating that brute-force sampling combined with intelligent filtering can compensate for the imprecision of individual model outputs, a principle that recurs throughout LLM-guided search.

The open-source community responded in May 2023 with StarCoder, a 15.5-billion parameter model developed by the BigCode collaboration and trained on one trillion tokens of permissively licensed code from The Stack, spanning over 80 programming languages [65]. StarCoder matched or exceeded the performance of OpenAI’s earlier code-cushman-001 model and achieved 40% pass@1 on HumanEval when appropriately prompted. Its release under an open license democratized access to capable code generation, enabling researchers to fine-tune and adapt code LLMs for specialized domains without dependence on proprietary APIs.

The generalist GPT-4 model, released in March 2023, further raised the ceiling. OpenAI’s technical report documented a 67% zero-shot pass@1 on HumanEval [66], representing a substantial jump over Codex and approaching the performance levels that previously required extensive sampling. GPT-4’s coding ability was further contextualized by SWE-bench, introduced in October 2023 as a benchmark of 2,294 real-world GitHub issues drawn from 12 popular Python repositories [67]. Unlike HumanEval’s self-contained function synthesis tasks, SWE-

bench requires understanding repository-level context, coordinating changes across multiple files, and reasoning about complex software dependencies. Initial evaluations found that even the best models solved fewer than 4% of issues, underscoring the gap between isolated code generation and genuine software engineering [67].

7.2. From Code Generation to Reward Synthesis

The capacity of LLMs to generate syntactically correct and semantically meaningful code opens a particularly consequential application in reinforcement learning. The reward function, which encodes the objective an RL agent seeks to optimize, is notoriously difficult to specify by hand. Poorly designed rewards lead to reward hacking, sparse feedback, or misaligned agent behavior. Traditionally, reward engineering has been a manual, iterative process requiring deep domain expertise.

Eureka, developed by Ma et al. at NVIDIA, first released in October 2023 and subsequently published at ICLR in May 2024, demonstrated that GPT-4 could serve as an autonomous reward designer [68]. The system operates through an evolutionary loop. GPT-4 generates candidate reward functions as executable Python code, each function is used to train an RL policy in simulation, and training statistics are fed back to the model as structured “reward reflection” summaries. GPT-4 then revises the reward functions based on this feedback, iterating until convergence. Across a suite of 29 open-source RL environments spanning 10 distinct robot morphologies, Eureka’s generated rewards outperformed expert human-engineered rewards on 83% of tasks, with an average normalized improvement of 52% [68]. The most striking demonstration involved dexterous pen spinning with an anthropomorphic Shadow Hand, a task so complex that no prior automated method had achieved it. Critically, Eureka required no task-specific prompting or pre-defined reward templates. The LLM’s pre-training on vast amounts of code and technical documentation endowed it with sufficient domain knowledge to reason about physical quantities, contact dynamics, and rotational mechanics.

A complementary approach, Language to Rewards by Yu et al. at DeepMind, released in June 2023 and presented at CoRL in November 2023, addressed the interface between natural language task descriptions and reward parameterization for robotic skill synthesis [69]. Rather than generating reward functions from scratch, this method translates human language instructions into reward parameters that configure a pre-existing reward structure, enabling non-expert users to specify complex robot behaviors through natural language. The system demonstrated capabilities including novel locomotion gaits and manipulation skills in MuJoCo environments, highlighting how LLMs can bridge the gap between human intent and the mathematical formalism that RL algorithms require [69].

7.3. The Key Insight and Its Limitations

These results collectively illuminate a central insight. LLMs trained on code encode substantial domain knowledge about what constitutes well-structured programs, reasonable physical heuristics, and effective optimization objectives. When deployed within an iterative search loop, they function not as random mutation operators but as informed search operators, proposing modifications grounded in patterns learned from millions of human-authored programs. This is fundamentally different from the random perturbations employed by traditional evolutionary algorithms or the uninformed sampling of early program synthesis methods.

However, LLM-based code generation carries significant limitations that temper enthusiasm with necessary caution. Code hallucination, where the model generates plausible-looking but functionally incorrect code, remains pervasive. Research has identified a taxonomy of halluci-

nation types spanning at least 12 specific categories, including references to non-existent APIs, incorrect algorithmic logic, and fabricated library functions [70]. The quality of generated code is highly sensitive to prompt formulation. Variations in prompt format, complexity, and the inclusion or omission of contextual information can substantially alter model outputs, making reproducibility a concern [70]. Most fundamentally, LLMs provide no formal guarantees of correctness. Unlike verified synthesis methods grounded in formal logic, neural code generation is inherently probabilistic, and the risk of subtle bugs that pass superficial testing but fail on edge cases is ever-present. For safety-critical applications, this limitation demands that LLM-generated code be treated as a starting point for human review rather than a finished product.

The inability to guarantee correctness is particularly relevant in the reinforcement learning context, where a subtly flawed reward function can produce policies that appear competent during training but exhibit catastrophic failures at deployment. The field therefore faces a productive tension. LLMs dramatically accelerate the search for good RL components, but the absence of formal verification means that their outputs must always be validated empirically, an observation that motivates the evolutionary frameworks discussed in subsequent sections.

8. LLM-Guided Evolutionary Search: FunSearch and AlphaEvolve

The convergence of large language models and evolutionary computation has produced a new class of search algorithms in which LLMs serve not as end-to-end solvers but as intelligent mutation operators within an evolutionary loop. Rather than prompting a model to output a solution directly, these systems evolve *programs* that encode solutions, leveraging the model’s capacity for code generation while relying on automated evaluators to supply rigorous fitness signals. This section traces the development of this paradigm from its conceptual foundations through its most recent extensions, highlighting FunSearch, AlphaEvolve, and several contemporaneous systems that collectively define the frontier of LLM-guided evolutionary program search.

8.1. Foundations: Evolution Through Large Models

The theoretical groundwork for using LLMs as evolutionary operators was laid by Lehman et al. in June 2022 [71], who observed that language models trained on large corpora of source code and version-control diffs can approximate the kinds of perturbations that human programmers apply when iteratively refining software. In their framework, dubbed Evolution through Large Models (ELM), the LLM replaces the hand-designed mutation and crossover operators of classical genetic programming with *intelligent perturbation*: given a piece of code and an optional natural-language commit message describing the intended change, a diff model proposes semantically meaningful modifications. When combined with MAP-Elites for diversity maintenance, ELM generated hundreds of thousands of functional Python programs producing locomoting robots in the Sodarace domain, a task that fell entirely outside the LLM’s pretraining distribution. The work demonstrated that LLMs could serve as open-ended variation operators, but left open the question of whether such systems could produce genuinely novel scientific or mathematical results.

8.2. FunSearch: Mathematical Discovery via Program Search

FunSearch, introduced by Romera-Paredes et al. and published in *Nature* in December 2023 [72], answered that question affirmatively. The system pairs a pretrained LLM (specifically, a Codey model) with a deterministic evaluator inside an island-based evolutionary loop. Crucially,

FunSearch searches in *function space* rather than solution space: instead of asking the LLM to output a cap set or a bin-packing assignment, it asks the model to write a Python function whose execution produces the desired object. This indirection is what makes the approach both scalable and verifiable, since the evaluator can check the output of any candidate program with certainty.

The evolutionary architecture employs an island model to balance exploration and exploitation. The population of programs is distributed across m independent islands, each initialized with a copy of a user-provided seed program. When constructing a prompt, the system uniformly samples an island, then selects $k = 2$ programs from within that island using a hierarchical scheme: programs are first clustered by their *signature* (a vector of scores on several benchmark inputs), and sampling favors clusters with higher scores while preferring shorter programs within each cluster. The two selected programs are arranged in ascending order of quality and presented to the LLM as successive “versions,” implicitly encouraging the model to generate a further improvement. Periodically, the lowest-performing islands are reset and re-seeded from higher-performing ones, preventing premature convergence while preserving the diversity that fuels continued exploration.

Applied to the cap set problem in extremal combinatorics, FunSearch discovered constructions of large cap sets that surpassed the best previously known results, including the largest improvement to the asymptotic lower bound in twenty years. In dimension $n = 8$, the system found a larger cap set from scratch, without requiring explicit knowledge of how to combine lower-dimensional solutions, as prior human-designed constructions had done. FunSearch also demonstrated practical generality by discovering online bin-packing heuristics that outperformed widely used baselines, with the evolved programs favoring tight-fit packing over the conventional best-fit strategy. The work was groundbreaking because it constituted the first demonstration that LLMs could contribute to genuinely novel mathematical discoveries, rather than merely recombining known results.

8.3. AlphaEvolve: From Functions to Full Programs

AlphaEvolve, unveiled by Google DeepMind in May 2025 [73], dramatically expanded the scope of LLM-guided evolutionary search by evolving *entire codebases* rather than individual functions. The system employs an ensemble of Gemini models: Gemini Flash maximizes the breadth of candidate solutions explored per unit of compute, while Gemini Pro provides deeper, more insightful suggestions for particularly promising evolutionary trajectories. An evolutionary database determines which programs are selected for future prompts, and automated evaluators verify correctness and measure fitness across potentially long-running test suites executed in parallel across many machines.

The headline result was a breakthrough in matrix multiplication. AlphaEvolve discovered an algorithm for multiplying 4×4 complex-valued matrices using only 48 scalar multiplications, improving upon Strassen’s algorithm from August 1969, which required 49 multiplications in this setting. Because matrix multiplication algorithms compose recursively, even a single multiplication saved at the 4×4 base case compounds into substantial savings at higher dimensions. Applied to more than 50 open problems in mathematics, AlphaEvolve rediscovered the state-of-the-art solution for 75% of them and found strictly better solutions for 20%.

Beyond pure mathematics, AlphaEvolve produced results deployed in production at Google. It discovered a scheduling heuristic for Borg, Google’s cluster workload management system, that has been running in production for over a year and recovers 0.7% of Google’s overall compute

capacity, allowing more tasks to execute on the same physical infrastructure. In hardware design, AlphaEvolve proposed modifications to Verilog code describing arithmetic circuits for matrix multiplication, eliminating redundancies that were subsequently integrated into a future Tensor Processing Unit design. It also optimized a matrix multiplication kernel used in Gemini model training, achieving a 23% speedup for that specific operation and reducing overall training time by approximately 1%. The breadth of these applications, spanning pure mathematics, systems software, hardware description languages, and machine learning infrastructure, underscores the generality that comes from evolving full programs rather than isolated functions.

8.4. Reflective and Adaptive Extensions

Several concurrent lines of work have sought to improve the sample efficiency and expressiveness of LLM-guided evolutionary search. ReEvo, introduced by Ye et al. in February 2024 on arXiv and published at NeurIPS 2024 in December 2024 [74], frames the LLM as a *hyper-heuristic* and augments the evolutionary loop with a reflective mechanism. Each generation proceeds through selection, short-term reflection, crossover, long-term reflection, and elitist mutation. The reflection steps prompt the LLM to analyze why certain heuristics outperformed others, producing what the authors term “verbal gradients” that steer subsequent generations toward more promising regions of the search space. Evaluated across six NP-hard combinatorial optimization problems, ReEvo produced state-of-the-art or competitive heuristics, evolutionary algorithms, and neural solvers while requiring fewer fitness evaluations than prior language-based hyper-heuristic methods.

EvoTune, proposed by Surina et al. in April 2025 on arXiv and published at COLM 2025 in October 2025 [75], addresses a fundamental limitation of all prior systems: they treat the LLM as a *static* generator, discarding the rich signal produced by the evolutionary process itself. EvoTune closes this loop by alternating between evolutionary search and online reinforcement learning. During the search phase, the system operates in the standard FunSearch-like manner, sampling from a program database organized into islands and generating candidates via LLM inference. Periodically, the system switches to an RL fine-tuning phase in which the LLM is updated via online Direct Preference Optimization (DPO), leveraging pairwise comparisons between stored programs to teach the model to propose higher-quality mutations. Evaluated on bin packing, the traveling salesman problem, and the flatpack problem using small open-weight models (Llama 3.2 1B, Phi 3.5 Mini, and Granite 3.1 2B), EvoTune consistently outperformed the static-LLM baseline, demonstrating that even modest models can become effective search operators when fine-tuned on their own evolutionary trajectory.

8.5. Evolving Game-Theoretic Algorithms

The most recent extension of this paradigm applies AlphaEvolve to multi-agent reinforcement learning (MARL) algorithm design. Li et al., posted to arXiv in February 2026 [76], used AlphaEvolve’s evolutionary framework to treat the source code of game-theoretic algorithms as a genome subject to LLM-driven semantic mutation. Rather than tuning hyperparameters or composing predefined building blocks, the system rewrites algorithmic logic, introduces new control flows, and injects novel symbolic operations. The search produced two previously unknown algorithm variants. Volatility-Adaptive Discounted CFR (VAD-CFR) dynamically adjusts its discounting parameters based on the volatility of regret updates, tracked via an exponential weighted moving average, and incorporates a hard warm-start that delays policy averaging until iteration 500. Smoothed Hybrid Optimistic Regret PSRO (SHOR-PSRO) mixes regret-based stability with aggressive greedy exploitation, annealing the balance over

the training horizon. VAD-CFR matched or surpassed state-of-the-art performance in 10 out of 11 imperfect-information game environments, including Leduc Poker and Liar’s Dice, with four-player Kuhn Poker as the sole exception. These results demonstrate that LLM-guided evolution can discover not only mathematical constructions and engineering heuristics but also fundamentally new algorithms in domains requiring strategic reasoning under uncertainty.

8.6. Synthesis

The trajectory from ELM through FunSearch, AlphaEvolve, ReEvo, EvoTune, and the MARL extension reveals a consistent pattern of increasing scope, autonomy, and integration. Each successive system has expanded the unit of evolution (from single functions to entire codebases), deepened the feedback loop (from static LLM queries to online fine-tuning), and broadened the domain of application (from mathematical curiosities to production infrastructure). The central insight unifying this line of work is that LLMs are most powerful not as oracles that produce answers, but as *search operators* whose generative capacity can be harnessed, evaluated, and iteratively refined within a principled optimization framework.

9. Evolving RL Algorithms: The GECCO 2026 Paper

Reinforcement learning algorithms are conventionally defined by hand-designed update rules that map experience to parameter changes. While automated discovery has been applied to adjacent components of RL systems, including neural architecture search [77] (ICLR, April 2017), hyperparameter tuning via meta-gradients [57] (NeurIPS, December 2018), and reward shaping with Eureka [69] (ICLR, May 2024), the learning update rule itself has remained largely fixed. A small number of prior works have explored learning the update rule directly. Oh et al. [60] optimized differentiable update functions at NeurIPS in December 2020, while Co-Reyes et al. [61] evolved structured symbolic loss expressions via genetic programming at ICLR in May 2021. However, these methods operate over restricted, parameterized representations of the learning rule, such as differentiable update networks or symbolic loss templates, and therefore cannot express fundamentally different training procedures as executable code. Sygkounas, Loutfi, and Persson [78] address this gap directly in their paper “Evolutionary Discovery of Reinforcement Learning Algorithms via Large Language Models,” accepted at GECCO 2026, scheduled for July 2026 in San José, Costa Rica. Their framework searches over complete, executable update rules by combining evolutionary computation with large language models as generative variation operators, producing novel RL algorithms that avoid all canonical mechanisms.

9.1. Problem Statement and Motivation

The central argument of the paper is that searching over RL update rules is fundamentally incompatible with gradient-based optimization. The search space consists of discrete program structures implementing full training procedures rather than continuous parameters. Even small changes to update logic can induce qualitatively different learning dynamics, and RL algorithms are notoriously sensitive to implementation details [38], [78], as shown in the PPO and TRPO case study presented at ICLR in April 2020. Hyperparameters that stabilize one update rule often fail for another [79], a finding first reported at AAAI in February 2018, which prevents effective local refinement. As a result, each candidate update rule must be treated as a distinct algorithm whose quality can only be assessed through complete training runs. Under these conditions, population-based evolutionary methods are a natural fit because they compare candidates using end-to-end performance without requiring gradients or smooth objective landscapes, as argued by the Population Based Training work released in November 2017 [16] and the Learned Policy Gradient framework of [60].

The work builds directly on REvolve [80], an evolutionary system first posted to arXiv in June 2024 and published at ICLR in April 2025 by the same research group at Orebro University, which uses large language models as generative variation operators for reward-function discovery. REvolve formulates reward optimization as an island-based evolutionary process, where LLMs generate syntactically valid reward code that is evaluated by training RL agents. Sygkounas et al. [78] apply the same evolutionary architecture to a fundamentally different and more challenging search space. Instead of evolving reward functions, which are relatively short expressions with graded behavioral effects, they evolve entire learning update rules as executable Python code. Each candidate defines a complete training procedure, including a differentiable loss function $\mathcal{L}_f(\theta, \xi_t; \mathcal{D}_t)$ and all auxiliary state updates, under a fixed policy architecture, optimizer, and training loop. In this formulation, each distinct update rule f defines a distinct RL algorithm.

9.2. The Evolutionary Loop

The evolutionary search follows an island-based population model [80]. For each generation, each island produces 24 new candidate update rules through two variation operators selected stochastically. Macro mutation, applied with probability $p_{\text{macro}} = 0.65$, rewrites exactly one semantically coherent component of an existing update rule in a single step. This choice reflects the finding that micro mutations, which work well for short reward expressions in REvolve, are insufficient for update rules where deficiencies in early generations are structural rather than local. Diversity-aware crossover, applied with probability $p_{\text{cross}} = 0.35$, combines two parent rules selected via a composite score that balances fitness-proportional selection with structural dissimilarity measured by normalized Levenshtein distance, following the parallel genetic algorithms survey of [81] published in 1998. The Levenshtein weight $\alpha = 0.5$ prevents crossover from recombining near-duplicate parents, which would collapse population diversity, using the edit distance introduced by [82] in 1966.

Both operators are implemented through a conditional generative model realized by a large language model, $f' \sim q_{\varphi}(f_1, f_2, \text{op}, \mathcal{R}, \mathcal{E})$, which receives the parent update rules, the selected operation type, summary training metrics, and environment interface specifications. The LLM then outputs a complete executable implementation of a new update rule. Critically, during evolutionary variation, the generation process is constrained to forbid explicit instantiation of canonical RL mechanisms. The search excludes actor-critic decomposition in the spirit of the novelty search work presented at GECCO in July 2011 [83], bootstrapped value targets, temporal-difference error computation in the tradition of the actor-critic formulation introduced at NeurIPS in December 1999 [84], and policy gradient methods. These constraints are enforced only during evolution, not during post-evolution analysis, and serve to force the emergence of genuinely nonstandard learning rules.

Each evolutionary run executes for 10 generations. Fitness evaluation is rigorous. Every candidate update rule is trained on a fixed set of five environments (CartPole-v1, MountainCar-v0, Acrobot-v1, LunarLander-v3, and HalfCheetah-v5) using 5 independent random seeds per environment, yielding 25 training runs per candidate. Performance is summarized by the maximum evaluation return achieved during training for each environment, then normalized to the unit interval using environment-specific reference bounds and averaged across environments to produce a single scalar fitness. A candidate is accepted into the island population only if its fitness exceeds the current population mean, and accepted candidates replace the lowest-fitness members.

9.3. LLMs as Variation Operators and the Performance Gap

The paper evaluates two large language models as generative operators: GPT-5.2 and Claude 4.5 Opus. For each model, two independent evolutionary runs with different random seeds are conducted. The results reveal a stark performance gap. GPT-5.2 consistently attains higher fitness values throughout evolution, with final-generation fitness in the range $[0.65, 0.69]$ across seeds. Claude 4.5 Opus achieves substantially lower fitness in the range $[0.37, 0.47]$ and does not produce competitive candidates in the final generation [78]. Both models exhibit monotonic increases in maximum fitness across generations, confirming that the evolutionary loop successfully improves candidate quality over time. However, the magnitude of the gap is notable: update rules evolved with Claude 4.5 Opus are excluded entirely from further analysis because they fail to reach competitive performance levels. This finding underscores that the choice of LLM as a variation operator materially affects the quality of the search,

and suggests that code generation capabilities for complex, tightly-coupled training procedures differ substantially across model families.

9.4. Post-Evolution Hyperparameter Optimization

The paper introduces a key extension over REvolve: a post-evolution LLM-guided hyperparameter optimization (LLM-HPO) stage. REvolve terminates after evolutionary search without additional tuning, but this is insufficient for update rules because RL algorithms are highly sensitive to internal scalar parameters [38], [78]. A fixed parameterization can severely underestimate the quality of an update rule. For each top- K evolved update rule $\hat{f}$, the LLM is provided with the full implementation and environment specifications and returns bounded numeric intervals $\mathcal{B}_{\hat{f}} = \prod_{j=1}^d [\ell_j, u_j]$ for each internal scalar parameter. A random sweep then uniformly samples parameter vectors from this region, evaluates fitness through full training runs, and retains the best-performing configuration β_i^* for final evaluation.

9.5. The Discovered Algorithms

The two highest-performing algorithms, both evolved using GPT-5.2, are Confidence-Guided Forward Policy Distillation (CG-FPD) and Differentiable Forward Confidence-Weighted Planning with Controllability Prior (DF-CWP-CP). Both are structurally novel in that they avoid value functions, policy gradients, and Bellman-style updates entirely, instead relying on planning-derived learning signals.

CG-FPD trains a policy by distilling short-horizon plans from a learned latent dynamics model. A latent encoder maps observations to a compact representation, and a learned dynamics model predicts transitions in latent space. At each update step, the cross-entropy method (CEM) generates candidate action sequences, which are scored by rolling out the dynamics model and accumulating predicted rewards. The best plan identified by CEM serves as a supervised teacher signal: the policy is updated via distillation to match the first action of the plan. No value function is learned, and no temporal-difference targets are computed. The entire learning signal derives from the quality of the planner’s output.

DF-CWP-CP takes a different approach to the same planning-without-values paradigm. It optimizes the policy via differentiable short-horizon world-model rollouts that incorporate confidence-weighted desirability and controllability objectives. A confidence head estimates the reliability of each predicted transition, and a dual-flow regularization mechanism maintains two parallel policy representations that are periodically reconciled. A controllability augmentation term encourages the agent to prefer states where its actions have high influence on future outcomes, biasing exploration toward regions of the state space where learning is most effective.

9.6. Empirical Results

The discovered algorithms are evaluated on ten Gymnasium environments, including the five used during evolution and five unseen environments (Walker2d, InvertedPendulum, Reacher, Swimmer, and Pusher), and compared against PPO, A2C, DQN, and SAC baselines using an identical 256×256 MLP policy architecture.

Environment	PPO	A2C	DQN	SAC	CG-FPD	DF-CWP-CP
CartPole	500.0	500.0	500.0	–	500.0	500.0
LunarLander	246.6	246.6	250.1	–	241.2	260.6
MountainCar	–128.1	–134.2	–147.8	–	–105.8	–108.7
Acrobot	–63.5	–213.6	–61.9	–	–90.6	–78.7
HalfCheetah	1579.1	795.9	–	4988.5	2407.9	2103.8
Reacher	–3.15	–6.48	–	–2.05	–2.67	–5.43
Swimmer	95.0	49.1	–	87.8	247.5	219.8
Inv. Pendulum	1000.0	1000.0	–	1000.0	1000.0	1000.0
Walker2d	3163.2	801.5	–	4595.3	1603.8	1297.6
Pusher	–25.5	–32.4	–	–25.5	–27.1	–28.7

Table 1: Maximum evaluation return (mean over 5 seeds) for evolved algorithms vs. baselines. Top five rows are training environments; bottom five are unseen. Bold indicates best performance. Dashes indicate inapplicable baselines (DQN for continuous, SAC for discrete). Adapted from Sygkounas et al. [78].

The results reveal a nuanced performance profile. On the unseen Swimmer environment, CG-FPD achieves a return of 247.5, dramatically outperforming PPO (95.0), SAC (87.8), and A2C (49.1). DF-CWP-CP also excels on Swimmer at 219.8. On LunarLander, DF-CWP-CP achieves the highest return (260.6) among all methods. Both evolved algorithms match the optimal score on CartPole and InvertedPendulum. On MountainCar, CG-FPD achieves -105.8 , outperforming all baselines including PPO (-128.1) and DQN (-147.8). However, the evolved algorithms fall short on high-dimensional continuous control tasks where SAC dominates. On HalfCheetah, SAC achieves 4988.5 compared to CG-FPD’s 2407.9, and on Walker2d, SAC reaches 4595.3 versus CG-FPD’s 1603.8. This pattern suggests that planning-based update rules without value functions scale less effectively to tasks requiring fine-grained continuous control over many degrees of freedom.

9.7. Training Stability and Ablation Studies

A notable limitation of the discovered algorithms is training instability. Evaluation curves show that both CG-FPD and DF-CWP-CP reach high performance within the training budget, but learning is often non-monotonic and performance is not consistently maintained at the end of training [78]. The reported results rely on best-checkpoint selection rather than final-policy performance, which masks the instability during training.

Two ablation studies provide additional insight. The first examines the Levenshtein regularization weight α in the crossover operator. With $\alpha = 0$ (no structural regularization), maximum fitness reaches approximately 0.50; with $\alpha = 1$ (full regularization), fitness reaches 0.56. The intermediate setting $\alpha = 0.5$ used in the main experiments achieves the highest fitness range of $[0.65, 0.69]$, demonstrating that partial structural regularization yields the best balance between preserving functional code structure and enabling exploratory variation.

The second ablation adds a terminal value bootstrap to CG-FPD via a value head trained with one-step temporal-difference learning, used only as a bounded terminal bonus in the planner score. Across all tested environments, adding this canonical RL mechanism reduces peak performance but consistently lowers variance across seeds. This result is particularly telling: it demonstrates that CG-FPD is not limited by missing long-horizon return estimates,

and that introducing learned value signals actually biases planning and degrades peak performance even when used conservatively. The stability-performance tradeoff suggests that the evolved algorithms occupy a fundamentally different design region than conventional value-based methods.

9.8. Computational Cost and Broader Implications

The computational requirements are substantial. All evolutionary experiments were conducted on four NVIDIA A100 GPUs with 40 GB of memory each, and a single evolutionary generation required approximately 30 hours of wall-clock time. The full evolutionary search (10 generations) therefore requires roughly 300 hours of GPU time per run, executed on the LUMI supercomputer. This cost is driven by the requirement that each candidate must be evaluated through 25 complete training runs (5 environments by 5 seeds).

The broader implication of this work is that LLM-guided evolutionary search can discover RL algorithms that are structurally distinct from all existing methods. The discovered algorithms replace the entire value-estimation and policy-gradient machinery of modern RL with planning-based distillation and differentiable world-model rollouts. While the current generation of evolved algorithms does not uniformly surpass established baselines, the framework demonstrates that the space of viable RL update rules extends well beyond what human designers have explored. As noted by the authors, the discovered algorithms arise from novel recombinations of existing RL mechanisms rather than from fundamentally new update forms [78], reflecting both the structure of the search space and the representational limits of current language models. Future improvements in LLM code generation capabilities and evaluation efficiency may yield algorithms that more consistently outperform hand-designed baselines across diverse environments.

10. Critical Analysis and Open Problems

The preceding sections have traced a compelling arc from classical evolutionary computation to LLM-guided discovery of reinforcement learning algorithms. The results are impressive on their face, but intellectual honesty demands that we interrogate the foundations, limitations, and unresolved tensions of this emerging paradigm. This section offers a systematic critical analysis organized around seven open problems, followed by a discussion of future directions that could address them.

10.1. The Novelty Question

The most fundamental question facing LLM-guided algorithm discovery is whether the outputs constitute genuine scientific contributions or sophisticated recombination. Sygkounas et al. [78] are admirably transparent on this point, acknowledging that “the discovered algorithms arise from novel recombinations of existing reinforcement-learning mechanisms rather than from fundamentally new update forms.” The evolved loss functions combine entropy regularization, clipped ratios, and advantage weighting in configurations not previously explored, but every individual component has antecedents in the RL literature.

This admission invites a sharper critique articulated by Ting [85] in *Nature Astronomy* in February 2025, who argues that if a large language model can readily replicate the core intellectual contribution of a scientific work, that contribution may lack sufficient novelty. Ting proposes what amounts to an “LLM replicability test” for scientific merit, suggesting that peer reviewers should evaluate whether a contribution lies meaningfully outside the model’s training

distribution. Under this criterion, loss functions assembled from known RL primitives by an LLM would fail to qualify as genuinely novel discoveries.

There is, however, a credible counter-argument rooted in the innovation studies literature. Schumpeter [86], writing in 1939, defined innovation as *neue Kombinationen* — new combinations of existing means of production. Fleming [87], writing in *Management Science* in January 2001, formalized this insight in his theory of recombinant search, demonstrating empirically that breakthrough inventions arise disproportionately from the combination of components not previously joined. Under this framework, the LLM functions as a combinatorial search engine operating over a space of algorithmic primitives, and the novelty lies in the specific combinations explored rather than in the primitives themselves. The tension between these perspectives remains unresolved and reflects a deeper philosophical disagreement about the nature of scientific contribution that predates the LLM era entirely.

10.2. The Compute Problem

Even if one accepts the scientific value of evolved algorithms, the practical barrier of computational cost is severe. In the Sygkounas et al. framework [78], each candidate algorithm requires 25 full training runs (5 environments multiplied by 5 random seeds) to obtain a reliable fitness estimate. With 24 new candidates generated per island per generation across 10 generations, the total evaluation budget runs into the thousands of complete RL training procedures. Each generation consumed approximately 30 hours of wall-clock time on four A100 GPUs with 40 GB of memory, and the work required access to the LUMI supercomputer through EuroHPC. FunSearch [73], published in *Nature* in December 2023, and AlphaEvolve [74], announced by Google DeepMind in May 2025, similarly depend on Google DeepMind-scale infrastructure, with AlphaEvolve orchestrating Gemini 2.0 Flash and Gemini 2.0 Pro models in tandem for candidate generation.

This creates an uncomfortable asymmetry in the research landscape. The methodology is, in principle, general-purpose, yet in practice it is accessible only to groups with supercomputer allocations or industry backing. The broader RL research community — particularly at smaller universities and in developing countries — cannot replicate these experiments, let alone extend them. If the most promising direction in automated algorithm design requires resources available to fewer than a dozen institutions worldwide, the field risks becoming a spectator sport rather than a participatory science.

10.3. The LLM Dependence Problem

Perhaps the most alarming finding in Sygkounas et al. [78] is the dramatic sensitivity of results to the choice of language model. GPT-5.2 achieved final fitness scores in the range of 0.65 to 0.69, while Claude 4.5 Opus produced scores of only 0.37 to 0.47, a gap so large that the Claude-evolved algorithms were excluded from further analysis entirely. This is not a minor implementation detail. It means that the scientific conclusions of the paper — which algorithms were discovered, which mechanisms proved effective, which combinations were fruitful — are contingent on the specific proprietary model used as the evolutionary operator.

This dependence creates three distinct problems. First, reproducibility is undermined because API-vended models are updated, versioned, and occasionally deprecated without guarantees of behavioral stability. An experiment conducted with GPT-5.2 in March 2026 may not yield the same results with GPT-5.2 in March 2027, or with whatever model succeeds it. Second, the science becomes entangled with commercial interests, as results that “only work with OpenAI’s API” are not fully open science. Third, the sensitivity itself is poorly understood. Why does

GPT-5.2 outperform Claude 4.5 Opus by such a wide margin on this task? Is it the code generation capability, the implicit knowledge of RL algorithms, or some artifact of the prompt format? Without answers to these questions, the LLM is a black box within a black box, and the evolved algorithms are doubly opaque.

10.4. The Generalization Problem

The evaluation protocol in Sygkounas et al. [78] trains evolved algorithms on five Gymnasium environments and tests on ten (five training plus five novel). The results reveal a characteristic pattern familiar from the meta-learning literature, including work by Yao et al. presented at ICML in July 2021 [88] and by Kirsch et al. at ICLR in April 2020 [59]. algorithms that are competitive on the training distribution do not reliably transfer. The best evolved algorithm, CG-FPD, achieved a score of 2,407.88 on HalfCheetah-v5, compared to SAC’s 4,988.5, and 1,603.80 on Walker2d-v5 versus SAC’s 4,595.30. These are not marginal shortfalls but substantial performance gaps on standard continuous control benchmarks. On the novel Swimmer environment, a different evolved algorithm (DF-CWP-CP) performed well, but it underperformed on Reacher and Walker2d.

This pattern echoes the broader challenge of meta-overfitting to task distributions that has been documented extensively in meta-reinforcement learning [88]. When algorithms are optimized for aggregate fitness across a fixed set of environments, the evolutionary pressure may exploit shared structure in those environments rather than discovering genuinely general learning principles. The analogy to overfitting in supervised learning is direct and troubling: just as a neural network can memorize training data, an evolutionary process can “memorize” a benchmark suite.

10.5. The Theory Vacuum

Classical reinforcement learning, for all its practical difficulties, rests on theoretical foundations. Q-learning converges to the optimal action-value function under well-defined conditions, as established by Watkins and Dayan in *Machine Learning* in May 1992 [20]. Policy gradient methods have known convergence properties, formalized by Konda and Tsitsiklis at NeurIPS in December 1999 [84]. Even actor-critic algorithms possess finite-time convergence guarantees under function approximation [84]. None of these guarantees exist for any LLM-evolved algorithm. There are no convergence proofs, no sample complexity bounds, and no formal characterization of the search space over which evolution operates.

This absence is not merely an aesthetic concern. Without theoretical grounding, practitioners have no basis for predicting when an evolved algorithm will succeed or fail, no way to diagnose failures systematically, and no guarantee that training will not diverge on a new task. The contrast with the classical paradigm is stark: SAC may underperform an evolved algorithm on three benchmarks, but at least its maximum entropy objective is understood analytically, its soft Bellman equation is well-defined, and its convergence behavior under mild assumptions is characterized by the Soft Actor-Critic analysis presented at ICML in July 2018 [31]. The evolved algorithms offer performance without understanding, which is a fragile bargain.

10.6. The Reproducibility Crisis

Henderson et al. [79], in work presented at AAAI in February 2018, documented the reproducibility crisis in deep RL, demonstrating that implementation details, hyperparameter choices, random seeds, and even codebases can produce dramatically different results for nominally identical algorithms. The LLM-guided evolution paradigm layers two additional sources

of stochasticity on top of these existing difficulties: the randomness of the evolutionary search process and the stochasticity inherent in LLM generation. The combination is formidable. Even if the LLM, the evolutionary parameters, the evaluation environments, and the random seeds are all held fixed, the temperature parameter in the LLM’s sampling introduces irreducible variation in the candidate programs generated.

Furthermore, the reporting conventions in this nascent field have not yet matured. When results are presented for the “best evolved algorithm” selected post hoc from a population of candidates across multiple evolutionary runs, the effective degrees of freedom in the analysis are enormous. This is analogous to the “best checkpoint” reporting that Henderson et al. [79] criticized in the broader deep RL literature, amplified by the additional selection pressure of choosing from an entire population of evolved candidates.

10.7. The Evaluation Bottleneck

Irpan [89], in an influential blog post from February 2018, argued persuasively that deep RL “doesn’t work yet,” cataloging the gap between impressive benchmark results and the requirements of real-world deployment. Eight years later, the situation has improved but the core critique retains force, particularly for LLM-evolved algorithms. The Gymnasium environments used for evaluation — CartPole, LunarLander, HalfCheetah, Walker2d — are the field’s standard benchmarks precisely because they are tractable and well-studied, but they are also stylized, low-dimensional, and far removed from production applications. An evolved algorithm that matches PPO on CartPole tells us very little about its behavior in robotics, autonomous driving, or large-scale recommendation systems.

The evaluation bottleneck is self-reinforcing. Because each candidate evaluation is expensive (as discussed in Section 10.2), researchers are incentivized to use the simplest environments that produce publishable results. This creates a feedback loop in which evolved algorithms are optimized for and validated on benchmarks that are insufficient to establish practical relevance, while the cost of more rigorous evaluation remains prohibitive.

10.8. Future Directions

Despite these criticisms, the paradigm is not without promise, and several research directions could address the limitations identified above. First, surrogate fitness models could dramatically reduce evaluation cost. The surrogate-assisted evolutionary algorithm literature, surveyed by Jin in *Swarm and Evolutionary Computation* in June 2011 [90], has demonstrated that learned approximations of the fitness function can reduce the number of expensive evaluations by orders of magnitude, and applying these techniques to algorithm evolution is a natural next step.

Second, the LLM dependence problem could be mitigated by the use of open-weight models. The OpenELM library, described in the *Handbook of Evolutionary Machine Learning* published in March 2024 [72], has already demonstrated that evolutionary search with LLMs need not require proprietary APIs, and the rapid improvement of open models such as Llama and Mistral suggests that competitive performance may soon be achievable without commercial dependencies.

Third, the theory vacuum could be partially addressed through formal verification of evolved algorithms. If the search space is constrained to produce algorithms with particular structural properties — for instance, algorithms expressible as contractive operators — then convergence guarantees could potentially be inherited from the structural constraints rather than proved de novo.

Fourth, the evaluation bottleneck calls for the development of richer benchmark suites designed specifically for algorithm discovery. These should include environments with different reward structures, observation spaces, and dynamics, explicitly testing transfer rather than in-distribution performance.

Finally, the most transformative possibility is the integration of LLM self-improvement during the evolutionary search, as explored in recent co-evolutionary approaches presented at COLM in October 2025 [76]. If the LLM itself can be fine-tuned on successful evolutionary discoveries, the search process becomes an engine of self-improvement rather than a static exploitation of pre-trained capabilities. Whether this leads to a genuine positive feedback loop or merely to faster convergence to the same recombinatory ceiling remains an open and fascinating question.

References

- [1] I. Rechenberg, *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Stuttgart: Frommann-Holzboog, 1973.
- [2] H.-P. Schwefel, *Numerical Optimization of Computer Models*. Chichester: Wiley, 1981.
- [3] H.-G. Beyer and H.-P. Schwefel, “Evolution Strategies: A Comprehensive Introduction,” *Natural Computing*, vol. 1, no. 1, pp. 3–52, Mar. 2002, doi: 10.1023/A:1015059928466.
- [4] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [5] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [6] K. A. D. Jong, “An Analysis of the Behavior of a Class of Genetic Adaptive Systems,” Doctoral dissertation, 1975.
- [7] D. B. Fogel, *Evolutionary Computation: Toward a New Philosophy of Machine Intelligence*. Piscataway, NJ: IEEE Press, 1995.
- [8] L. J. Fogel, A. J. Owens, and M. J. Walsh, *Artificial Intelligence through Simulated Evolution*. New York: John Wiley, 1966.
- [9] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA: MIT Press, 1992.
- [10] T. Back, *Evolutionary Algorithms in Theory and Practice*. New York: Oxford University Press, 1996.
- [11] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*. Berlin: Springer-Verlag, 2003.
- [12] K. O. Stanley and R. Miikkulainen, “Evolving Neural Networks through Augmenting Topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, June 2002, doi: 10.1162/106365602320169811.
- [13] K. O. Stanley, D. B. D'Ambrosio, and J. Gauci, “A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks,” *Artificial Life*, vol. 15, no. 2, pp. 185–212, Apr. 2009, doi: 10.1162/artl.2009.15.2.15202.
- [14] R. Miikkulainen *et al.*, “Evolving Deep Neural Networks,” *Artificial Intelligence in the Age of Neural Networks and Brain Computing*. Academic Press, pp. 293–312, 2019. [Online]. Available: <https://arxiv.org/abs/1703.00548>
- [15] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution Strategies as a Scalable Alternative to Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1703.03864>
- [16] M. Jaderberg *et al.*, “Population Based Training of Neural Networks.” [Online]. Available: <https://arxiv.org/abs/1711.09846>
- [17] E. Real, C. Liang, D. R. So, and Q. V. Le, “AutoML-Zero: Evolving Machine Learning Algorithms from Scratch,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, July 2020, pp. 8007–8019. [Online]. Available: <https://arxiv.org/abs/2003.03384>

- [18] R. S. Sutton, “Learning to Predict by the Methods of Temporal Differences,” *Machine Learning*, vol. 3, pp. 9–44, Aug. 1988, doi: 10.1007/BF00115009.
- [19] C. J. C. H. Watkins, “Learning from Delayed Rewards,” Doctoral dissertation, 1989.
- [20] C. J. C. H. Watkins and P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, pp. 279–292, May 1992, doi: 10.1007/BF00992698.
- [21] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [22] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy Gradient Methods for Reinforcement Learning with Function Approximation,” in *Advances in Neural Information Processing Systems 12 (NeurIPS)*, Dec. 1999, pp. 1057–1063. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf
- [23] V. Mnih *et al.*, “Playing Atari with Deep Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1312.5602>
- [24] V. Mnih *et al.*, “Human-Level Control through Deep Reinforcement Learning,” *Nature*, vol. 518, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.
- [25] L.-J. Lin, “Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching,” *Machine Learning*, vol. 8, pp. 293–321, May 1992, doi: 10.1007/BF00992699.
- [26] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The Arcade Learning Environment: An Evaluation Platform for General Agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, June 2013, doi: 10.1613/jair.3912.
- [27] V. Mnih *et al.*, “Asynchronous Methods for Deep Reinforcement Learning,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, June 2016, pp. 1928–1937. [Online]. Available: <https://arxiv.org/abs/1602.01783>
- [28] T. P. Lillicrap *et al.*, “Continuous Control with Deep Reinforcement Learning,” in *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, May 2016. [Online]. Available: <https://arxiv.org/abs/1509.02971>
- [29] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, July 2018, pp. 1587–1596. [Online]. Available: <https://arxiv.org/abs/1802.09477>
- [30] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2012, pp. 5026–5033. doi: 10.1109/IROS.2012.6386109.
- [31] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, July 2018, pp. 1861–1870. [Online]. Available: <https://arxiv.org/abs/1801.01290>
- [32] T. Haarnoja *et al.*, “Soft Actor-Critic Algorithms and Applications.” [Online]. Available: <https://arxiv.org/abs/1812.05905>

- [33] Y. Tassa *et al.*, “DeepMind Control Suite.” [Online]. Available: <https://arxiv.org/abs/1801.00690>
- [34] S. Kakade and J. Langford, “Approximately Optimal Approximate Reinforcement Learning,” in *Proceedings of the 19th International Conference on Machine Learning (ICML)*, July 2002, pp. 267–274. [Online]. Available: <https://dl.acm.org/doi/10.5555/645531.656005>
- [35] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, “Trust Region Policy Optimization,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, July 2015, pp. 1889–1897. [Online]. Available: <https://arxiv.org/abs/1502.05477>
- [36] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” in *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, May 2016. [Online]. Available: <https://arxiv.org/abs/1506.02438>
- [37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms.” [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [38] L. Engstrom *et al.*, “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO,” in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, Apr. 2020. [Online]. Available: <https://arxiv.org/abs/2005.12729>
- [39] S. Huang, R. F. J. Dossa, A. Raffin, A. Kanervisto, and W. Wang, “The 37 Implementation Details of Proximal Policy Optimization,” *ICLR Blog Track*, Mar. 2022, [Online]. Available: <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>
- [40] Z. Shao *et al.*, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.” [Online]. Available: <https://arxiv.org/abs/2402.03300>
- [41] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [42] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, and others, “DAPO: An Open-Source LLM Reinforcement Learning System at Scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2025. [Online]. Available: <https://arxiv.org/abs/2503.14476>
- [43] L. C. Melo, A. Abate, and Y. Gal, “Stabilizing Policy Gradients for Sample-Efficient Reinforcement Learning in LLM Reasoning,” in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, Apr. 2026. [Online]. Available: <https://arxiv.org/abs/2502.20344>
- [44] P. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep Reinforcement Learning from Human Preferences,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2017. [Online]. Available: <https://arxiv.org/abs/1706.03741>
- [45] N. Stiennon *et al.*, “Learning to Summarize from Human Feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2020. [Online]. Available: <https://arxiv.org/abs/2009.01325>
- [46] L. Ouyang *et al.*, “Training Language Models to Follow Instructions with Human Feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>

- [47] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2023. [Online]. Available: <https://arxiv.org/abs/2305.18290>
- [48] M. Azar *et al.*, “A General Theoretical Paradigm to Understand Learning from Human Preferences,” in *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, May 2024. [Online]. Available: <https://arxiv.org/abs/2310.12036>
- [49] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “KTO: Model Alignment as Prospect Theoretic Optimization,” in *Proceedings of the International Conference on Machine Learning (ICML)*, July 2024. [Online]. Available: <https://arxiv.org/abs/2402.01306>
- [50] Y. Meng, M. Xia, and D. Chen, “SimPO: Simple Preference Optimization with a Reference-Free Reward,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2405.14734>
- [51] Y. Bai *et al.*, “Constitutional AI: Harmlessness from AI Feedback.” [Online]. Available: <https://arxiv.org/abs/2212.08073>
- [52] N. Lambert *et al.*, “Tulu 3: Pushing Frontiers in Open Language Model Post-Training.” [Online]. Available: <https://arxiv.org/abs/2411.15124>
- [53] L. Gao, J. Schulman, and J. Hilton, “Scaling Laws for Reward Model Overoptimization,” in *Proceedings of the International Conference on Machine Learning (ICML)*, July 2023. [Online]. Available: <https://arxiv.org/abs/2210.10760>
- [54] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, “RL²: Fast Reinforcement Learning via Slow Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1611.02779>
- [55] J. X. Wang *et al.*, “Learning to Reinforcement Learn.” [Online]. Available: <https://arxiv.org/abs/1611.05763>
- [56] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, Aug. 2017, pp. 1126–1135. [Online]. Available: <https://arxiv.org/abs/1703.03400>
- [57] Z. Xu, H. P. van Hasselt, and D. Silver, “Meta-Gradient Reinforcement Learning,” in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, Dec. 2018. [Online]. Available: <https://arxiv.org/abs/1805.09801>
- [58] R. Houthoofd *et al.*, “Evolved Policy Gradients,” in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, Dec. 2018. [Online]. Available: <https://arxiv.org/abs/1802.04821>
- [59] L. Kirsch, S. van Steenkiste, and J. Schmidhuber, “Improving Generalization in Meta Reinforcement Learning using Learned Objectives,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Apr. 2020. [Online]. Available: <https://arxiv.org/abs/1910.04098>
- [60] J. Oh *et al.*, “Discovering Reinforcement Learning Algorithms,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, Dec. 2020. [Online]. Available: <https://arxiv.org/abs/2007.08794>

- [61] J. D. Co-Reyes *et al.*, “Evolving Reinforcement Learning Algorithms,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, May 2021. [Online]. Available: <https://arxiv.org/abs/2101.03958>
- [62] J. Oh *et al.*, “Discovering State-of-the-Art Reinforcement Learning Algorithms,” *Nature*, Oct. 2025, doi: 10.1038/s41586-025-09761-x.
- [63] J. Parker-Holder *et al.*, “Automated Reinforcement Learning (AutoRL): A Survey and Open Problems,” *Journal of Artificial Intelligence Research*, vol. 74, pp. 517–568, June 2022, doi: 10.1613/jair.1.13596.
- [64] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code.” [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [65] Y. Li *et al.*, “Competition-Level Code Generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, Dec. 2022, doi: 10.1126/science.abq1158.
- [66] R. Li *et al.*, “StarCoder: May the Source Be with You!” [Online]. Available: <https://arxiv.org/abs/2305.06161>
- [67] OpenAI, “GPT-4 Technical Report.” [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [68] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, May 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [69] Y. J. Ma *et al.*, “Eureka: Human-Level Reward Design via Coding Large Language Models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, May 2024. [Online]. Available: <https://arxiv.org/abs/2310.12931>
- [70] W. Yu *et al.*, “Language to Rewards for Robotic Skill Synthesis,” in *Proceedings of the Conference on Robot Learning (CoRL)*, Nov. 2023. [Online]. Available: <https://arxiv.org/abs/2306.08647>
- [71] F. Liu *et al.*, “Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code.” [Online]. Available: <https://arxiv.org/abs/2404.00971>
- [72] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh, and K. O. Stanley, “Evolution through Large Models,” *Handbook of Evolutionary Machine Learning*. Springer, Singapore, pp. 331–366, Nov. 2024. [Online]. Available: <https://arxiv.org/abs/2206.08896>
- [73] B. Romera-Paredes *et al.*, “Mathematical Discoveries from Program Search with Large Language Models,” *Nature*, vol. 625, pp. 468–475, Jan. 2024, doi: 10.1038/s41586-023-06924-6.
- [74] A. Novikov, M. Balog, N. Vu, and others, “AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery.” [Online]. Available: <https://arxiv.org/abs/2506.13131>
- [75] H. Ye *et al.*, “ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2402.01145>
- [76] A. Surina *et al.*, “Algorithm Discovery with LLMs: Evolutionary Search Meets Reinforcement Learning,” in *Proceedings of the Conference on Language Modeling (COLM)*, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2411.11612>

- [77] B. Zoph and Q. V. Le, “Neural Architecture Search with Reinforcement Learning,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Apr. 2017. [Online]. Available: <https://arxiv.org/abs/1611.01578>
- [78] A. Sygkounas, A. Loutfi, and A. Persson, “Evolutionary Discovery of Reinforcement Learning Algorithms via Large Language Models.” [Online]. Available: <https://arxiv.org/abs/2603.28416>
- [79] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep Reinforcement Learning that Matters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, Feb. 2018. [Online]. Available: <https://arxiv.org/abs/1709.06560>
- [80] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, “REvolve: Reward Evolution with Large Language Models Using Human Feedback,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2406.01309>
- [81] E. Cantu-Paz, “A Survey of Parallel Genetic Algorithms,” *Calculateurs Paralleles, Re-seaux et Systemes Repartis*, 1998, [Online]. Available: <https://www.lri.fr/~marc/EvoStar/cantu-paz98survey.pdf>
- [82] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” *Soviet Physics Doklady*, 1966, [Online]. Available: <https://nymity.ch/sybilhunting/pdf/Levenshtein1966a.pdf>
- [83] J. Lehman and K. O. Stanley, “Evolving a Diversity of Virtual Creatures Through Novelty Search and Local Competition,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, July 2011. doi: 10.1145/2001576.2001606.
- [84] V. R. Konda and J. N. Tsitsiklis, “Actor-Critic Algorithms,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Dec. 1999. [Online]. Available: <https://proceedings.neurips.cc/paper/1999/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [85] Y.-S. Ting, “Artificial Intelligence Compels the Astronomy Community to Rethink Research Identity and Redefine Excellence,” *Nature Astronomy*, vol. 9, pp. 317–318, Feb. 2025, doi: 10.1038/s41550-025-02505-9.
- [86] J. A. Schumpeter, *Business Cycles: A Theoretical, Historical, and Statistical Analysis of the Capitalist Process*. McGraw-Hill, 1939. [Online]. Available: <https://archive.org/details/in.ernet.dli.2015.223779>
- [87] L. Fleming, “Recombinant Uncertainty in Technological Search,” *Management Science*, vol. 47, no. 1, pp. 117–132, Jan. 2001, doi: 10.1287/mnsc.47.1.117.10671.
- [88] H. Yao *et al.*, “Improving Generalization in Meta-Learning via Task Augmentation,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, July 2021. [Online]. Available: <https://arxiv.org/abs/2007.14264>
- [89] A. Irpan, “Deep Reinforcement Learning Doesn't Work Yet.” [Online]. Available: <https://www.alexirpan.com/2018/02/14/rl-hard.html>
- [90] Y. Jin, “Surrogate-Assisted Evolutionary Computation: Recent Advances and Future Challenges,” *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, June 2011, doi: 10.1016/j.swevo.2011.05.001.